

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Raudatul Sholehah

NIM. 2310817220002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Raudatul Sholehah
NIM : 2310817220002

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program	42
C. Pembahasan	46
D. Tautan Git	65

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Tampilan List Jawaban Soal 1	42
Gambar 2. Screenshot Hasil Tampilan Detail Jawaban Soal 1	42
Gambar 3. Screenshot Hasil Tampilan Kunjungi Jawaban Soal 1	43
Gambar 4. Screenshot Hasil Tampilan Favorit Jawaban Soal 1.....	43
Gambar 5. Screenshot Hasil Tampilan Tanpa Koneksi Jawaban Soal	44
Gambar 6. Screenshot Hasil Tampilan List (landscape) Jawaban Soal 1	44
Gambar 7. Screenshot Hasil Tampilan Detail (landscape) Jawaban Soal 1	45
Gambar 8. Screenshot Hasil Tampilan Kunjungi (landscape) Jawaban Soal 1.....	45
Gambar 9. Screenshot Hasil Tampilan Favorit (landscape) Jawaban Soal 1	45
Gambar 10. Screenshot Hasil Tampilan Tanpa Internet (landscape) Jawaban Soal 1	45

DAFTAR TABEL

Tabel 1. Source Code AndroidManifest.xml Jawaban Soal 1	7
Tabel 2. Source Code MakeupApp.kt Jawaban Soal 1	7
Tabel 3. Source Code MakeupDao.kt Jawaban Soal 1	8
Tabel 4. Source Code MakeupEntity.kt Jawaban Soal 1	9
Tabel 5. Source Code MakeupMapper.kt Jawaban Soal 1	9
Tabel 6. Source Code DataStoreManager.kt Jawaban Soal 1	10
Tabel 7. Source Code MakeupDatabase.kt Jawaban Soal 1	11
Tabel 8. Source Code MakeupProduct.kt Jawaban Soal 1	11
Tabel 9. Source Code MakeupApiService.kt Jawaban Soal 1	11
Tabel 10. Source Code RetrofitInstance.kt Jawaban Soal 1	12
Tabel 11. Source Code MakeupRepository.kt Jawaban Soal 1	13
Tabel 12. Source Code ProductItem.kt Jawaban Soal 1	17
Tabel 13. Source Code Navigation.kt Jawaban Soal 1	17
Tabel 14. Source Code ApiMakeupItem.kt Jawaban Soal 1	20
Tabel 15. Source Code MakeupItem.kt Jawaban Soal 1	23
Tabel 16. Source Code MakeupListScreen.kt Jawaban Soal 1	27
Tabel 17. Source Code MakeupDetailScreen.kt Jawaban Soal 1	31
Tabel 18. Source Code MakeupViewModel.kt Jawaban Soal 1	34
Tabel 19. Source Code MakeupViewModelFactory.kt Jawaban Soal 1	34
Tabel 20. Source Code MainActivity.kt Jawaban Soal 1	36
Tabel 21. Source Code Color.kt Jawaban Soal 1	36
Tabel 22. Source Code Theme.kt Jawaban Soal 1	37
Tabel 23. Source Code Type.kt Jawaban Soal 1	38
Tabel 24. Source Code ConnectivityObserver.kt Jawaban Soal 1	39
Tabel 25. Source Code build.gradle.kts (Module App) Jawaban Soal 1	41

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- Gunakan caching strategy pada Room..
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya.

A. Source Code

1. AndroidManifest.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
3
4     <uses-permission
5 android:name="android.permission.INTERNET" />
6     <uses-permission
7 android:name="android.permission.ACCESS_NETWORK_STATE" />
8
9     <application
10 android:name=".MakeupApp"
11     android:allowBackup="true"
12     android:dataExtractionRules="@xml/data_extraction_rules"
13     android:fullBackupContent="@xml/backup_rules"
14     android:icon="@mipmap/ic_launcher"
15     android:label="@string/app_name"
16     android:roundIcon="@mipmap/ic_launcher_round"
17     android:supportsRtl="true"
18     android:theme="@style/Theme.Modul5"/>
```

18	tools:targetApi="31"
19	android:usesCleartextTraffic="true">
20	
21	<activity
22	android:name=".presentation.MainActivity"
23	android:exported="true"
24	android:label="@string/app_name"
25	android:theme="@style/Theme.Modul5">
26	<intent-filter>
27	<action
28	android:name="android.intent.action.MAIN" />
29	<category
30	android:name="android.intent.category.LAUNCHER" />
31	</intent-filter>
32	</activity>
33	</application>
34	</manifest>

Tabel 1. Source Code AndroidManifest.xml Jawaban Soal 1

2. MakeupApp.kt

1	package com.example.modul5
2	
3	import android.app.Application
4	import com.example.modul5.data.local.MakeupDatabase
5	import com.example.modul5.data.remote.RetrofitInstance
6	import com.example.modul5.data.repository.MakeupRepository
7	import com.example.modul5.util.ConnectivityObserver
8	import com.example.modul5.util.NetworkConnectivityObserver
9	
10	class MakeupApp : Application() {
11	private val database by lazy {
12	MakeupDatabase.getDatabase(this) }
13	val repository by lazy {
14	MakeupRepository(
15	apiService = RetrofitInstance.api,
16	makeupDao = database.makeupDao()
17	)
18	}
19	val connectivityObserver: ConnectivityObserver by lazy {
20	NetworkConnectivityObserver(applicationContext)
21	}
22	}

Tabel 2. Source Code MakeupApp.kt Jawaban Soal 1

3. MakeupDao.kt

1	package com.example.modul5.data.local.dao
2	
3	import androidx.room.Dao

```

4 import androidx.room.Query
5 import androidx.room.Upsert
6 import com.example.modul5.data.local.entity.MakeupEntity
7 import kotlinx.coroutines.flow.Flow
8
9 @Dao
10 interface MakeupDao {
11     @Upsert
12     suspend fun upsertAll(products: List<MakeupEntity>)
13
14     @Query("SELECT * FROM makeup_products ORDER BY id ASC")
15     fun getAllMakeup(): Flow<List<MakeupEntity>>
16
17     @Query("SELECT * FROM makeup_products WHERE id = :id")
18     fun getMakeupById(id: Int): Flow<MakeupEntity?>
19
20     @Query("UPDATE makeup_products SET isFavorite =
21 :isFavorite WHERE id = :id")
22     suspend fun updateFavoriteStatus(id: Int, isFavorite:
23 Boolean)
24
25     @Query("SELECT * FROM makeup_products WHERE isFavorite =
26 1 ORDER BY id ASC")
27     fun getFavoriteMakeup(): Flow<List<MakeupEntity>>
28
29     @Query("SELECT id FROM makeup_products WHERE isFavorite =
30 1")
31     suspend fun getFavoriteIds(): List<Int>
32
33     @Query("DELETE FROM makeup_products")
34     suspend fun clearAll()
35 }

```

Tabel 3. Source Code MakeupDao.kt Jawaban Soal 1

4. MakeupEntity.kt

```

1 package com.example.modul5.data.local.entity
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "makeup_products")
7 data class MakeupEntity(
8     @PrimaryKey val id: Int,
9     val brand: String?,
10    val name: String?,
11    val price: String?,
12    val imageLink: String?,
13    val directImageLink: String?,
14    val description: String?,
15    val productType: String?,
16    val isFavorite: Boolean

```

17	)
----	---

Tabel 4. Source Code MakeupEntity.kt Jawaban Soal 1

5. MakeupMapper.kt

1	package com.example.modul5.data.local.entity
2	
3	import com.example.modul5.data.model.MakeupProduct
4	
5	fun List<MakeupProduct>.toEntity(favoriteIds: List<Int>):
6	List<MakeupEntity> {
7	return this.map { apiProduct ->
8	MakeupEntity(
9	id = apiProduct.id,
10	brand = apiProduct.brand,
11	name = apiProduct.name,
12	price = apiProduct.price,
13	imageLink = apiProduct.imageLink,
14	directImageLink = apiProduct.directImageLink,
15	description = apiProduct.description,
16	productType = apiProduct.productType,
17	isFavorite = favoriteIds.contains(apiProduct.id)
18	)
19	}
20	}
21	fun MakeupEntity.toModel(): MakeupProduct {
22	return MakeupProduct(
23	id = this.id,
24	brand = this.brand,
25	name = this.name,
26	price = this.price,
27	imageLink = this.imageLink,
28	directImageLink = this.directImageLink,
29	description = this.description,
30	productType = this.productType,
31	isFavorite = this.isFavorite
32	)
33	}
34	
35	fun List<MakeupEntity>.toModel(): List<MakeupProduct> {
36	return this.map { it.toModel() }
37	}

Tabel 5. Source Code MakeupMapper.kt Jawaban Soal 1

6. DataStoreManager.kt

1	package com.example.modul5.data.local.preferences
2	
3	import android.content.Context
4	import androidx.datastore.preferences.core.*
5	import androidx.datastore.preferences.preferencesDataStore

6	import kotlinx.coroutines.flow.Flow
7	import kotlinx.coroutines.flow.map
8	
9	val Context.dataStore by preferencesDataStore(name = "settings")
10	
11	class DataStoreManager(private val context: Context) {
12	companion object {
13	val DARK_MODE_KEY =
14	booleanPreferencesKey("dark_mode")
15	}
16	val darkMode: Flow<Boolean> = context.dataStore.data
17	.map { prefs -> prefs[DARK_MODE_KEY] ?: false }
18	
19	suspend fun setDarkMode(enabled: Boolean) {
20	context.dataStore.edit { settings ->
21	settings[DARK_MODE_KEY] = enabled
22	}
23	}
24	}

Tabel 6. Source Code DataStoreManager.kt Jawaban Soal 1

7. MakeupDatabase.kt

1	package com.example.modul5.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	import com.example.modul5.data.local.dao.MakeupDao
8	import com.example.modul5.data.local.entity.MakeupEntity
9	
10	@Database(entities = [MakeupEntity::class], version = 2,
11	exportSchema = false)
12	abstract class MakeupDatabase : RoomDatabase() {
13	abstract fun makeupDao(): MakeupDao
14	
15	companion object {
16	@Volatile
17	private var INSTANCE: MakeupDatabase? = null
18	
19	fun getDatabase(context: Context): MakeupDatabase {
20	return INSTANCE ?: synchronized(this) {
21	val instance = Room.databaseBuilder(
22	context.applicationContext,
23	MakeupDatabase::class.java,
24	"makeup_database"
25	)
26	.fallbackToDestructiveMigration()
	.build()

27	INSTANCE = instance
28	instance
29	}
30	}
31	}
32	}

Tabel 7. Source Code MakeupDatabase.kt Jawaban Soal 1

8. MakeupProduct.kt

1	package com.example.modul5.data.model
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class MakeupProduct(
8	val id: Int,
9	val brand: String?,
10	val name: String?,
11	val price: String?,
12	@SerialName("image_link") val imageLink: String?,
13	@SerialName("direct_image_link") val directImageLink:
	String?,
14	val description: String?,
15	@SerialName("product_type") val productType: String?,
16	val isFavorite: Boolean = false
17	)

Tabel 8. Source Code MakeupProduct.kt Jawaban Soal 1

9. MakeupApiService.kt

1	package com.example.modul5.data.remote.api
2	
3	import com.example.modul5.data.model.MakeupProduct
4	import retrofit2.http.GET
5	
6	interface MakeupApiService {
7	@GET("api/v1/products.json")
8	suspend fun getProducts(): List<MakeupProduct>
9	}

Tabel 9. Source Code MakeupApiService.kt Jawaban Soal 1

10. RetrofitInstance.kt

1	package com.example.modul5.data.remote
2	
3	import com.example.modul5.data.remote.api.MakeupApiService
4	import
	com.jakewharton.retrofit2.converter.kotlinx.serialization.asC
	onverterFactory
5	import kotlinx.serialization.json.Json
6	import okhttp3.MediaType.Companion.toMediaType

```

7 import okhttp3.OkHttpClient
8 import okhttp3.logging.HttpLoggingInterceptor
9 import retrofit2.Retrofit
10
11 object RetrofitInstance {
12     private val json = Json {
13         ignoreUnknownKeys = true
14         coerceInputValues = true
15     }
16
17     private val logging = HttpLoggingInterceptor().apply {
18         level = HttpLoggingInterceptor.Level.BODY
19     }
20
21     private val client = OkHttpClient.Builder()
22         .addInterceptor(logging)
23         .build()
24
25     private val retrofit by lazy {
26         Retrofit.Builder()
27             .baseUrl("https://tugas-modul5-
atuls.free.beeceptor.com")
28             .client(client)
29             .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
30             .build()
31     }
32
33     val api: MakeupApiService by lazy {
34         retrofit.create(MakeupApiService::class.java)
35     }
36 }

```

Tabel 10. Source Code RetrofitInstance.kt Jawaban Soal 1

11. MakeupRepository.kt

```

1 package com.example.modul5.data.repository
2
3 import com.example.modul5.data.local.dao.MakeupDao
4 import com.example.modul5.data.local.entity.toEntity
5 import com.example.modul5.data.local.entity.toModel
6 import com.example.modul5.data.model.MakeupProduct
7 import com.example.modul5.data.remote.api.MakeupApiService
8 import kotlinx.coroutines.flow.Flow
9 import kotlinx.coroutines.flow.flow
10 import kotlinx.coroutines.flow.map
11 import java.io.IOException
12
13 class MakeupRepository(
14     private val apiService: MakeupApiService,
15     private val makeupDao: MakeupDao
16 ) {
17     fun fetchAndCacheProducts(): Flow<Result<Unit>> = flow {
18         try {
19             val networkResponse = apiService.getProducts()
20             val favoriteIds = makeupDao.getFavoriteIds()
21
22             makeupDao.upsertAll(networkResponse.toEntity(favoriteIds))
23             emit(Result.success(Unit))
24         } catch (e: IOException) {
25             emit(Result.failure(e))
26         } catch (e: Exception) {
27             emit(Result.failure(e))
28         }
29     }
30
31     fun getProductsFromCache(): Flow<List<MakeupProduct>> {
32         return makeupDao.getAllMakeup().map { it.toModel() }
33     }
34
35     fun getProductById(id: Int): Flow<MakeupProduct?> {
36         return makeupDao.getMakeupById(id).map {
37             it?.toModel() }
38     }
39
40     fun getFavoriteProducts(): Flow<List<MakeupProduct>> {
41         return makeupDao.getFavoriteMakeup().map {
42             it.toModel() }
43     }
44
45     suspend fun toggleFavoriteStatus(id: Int, isFavorite: Boolean) {
46         makeupDao.updateFavoriteStatus(id, isFavorite)
47     }
48 }

```

Tabel 11. Source Code MakeupRepository.kt Jawaban Soal 1

12. ProductItem.kt

```

1 package com.example.modul5.presentation.components
2
3 import androidx.compose.foundation.clickable
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.foundation.shape.RoundedCornerShape
6 import androidx.compose.material.icons.Icons
7 import androidx.compose.material.icons.filled.Favorite
8 import
9 androidx.compose.material.icons.outlined.FavoriteBorder
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.layout.ContentScale
16 import androidx.compose.ui.platform.LocalContext
17 import androidx.compose.ui.res.painterResource
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.text.style.TextOverflow
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.ui.unit.sp
22 import coil.compose.AsyncImage
23 import coil.request.ImageRequest
24 import com.example.modul5.R
25 import com.example.modul5.data.model.MakeupProduct
26 import java.text.NumberFormat
27 import java.util.Locale
28
29 fun formatPrice(price: String?): String {
30     return try {
31         val value = price?.toDoubleOrNull() ?: 0.0
32         val formatter =
33             NumberFormat.getCurrencyInstance(Locale("id", "ID"))
34             formatter.maximumFractionDigits = 0
35             formatter.format(value).replace("Rp", "Rp ")
36     } catch (e: Exception) { "Rp -" }
37 }
38
39 @Composable
40 fun ProductItem(
41     product: MakeupProduct,
42     onVisitClick: (String?) -> Unit,
43     onDetailClick: (Int) -> Unit,
44     onToggleFavorite: (Int) -> Unit,
45     isOnline: Boolean,
46     modifier: Modifier = Modifier
47 ) {
48     Card(
49         modifier = modifier
50             .fillMaxWidth()
51             .clickable(enabled = isOnline) {
52                 onDetailClick(product.id) },

```

```

50         shape = RoundedCornerShape(20.dp),
51         elevation =
CardDefaults.cardElevation(defaultElevation = 6.dp),
52         colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
53     ) {
54         Box {
55             Row(
56                 modifier =
Modifier.height(IntrinsicSize.Max),
57                 verticalAlignment = Alignment.Top
58             ) {
59                 AsyncImage(
60                     model =
ImageRequest.Builder(LocalContext.current).data(product.directImageLink).crossfade(true).build(),
61                     placeholder = painterResource(id =
R.drawable.ic_launcher_foreground),
62                     error = painterResource(id =
R.drawable.ic_launcher_foreground),
63                     contentDescription = product.name,
64                     contentScale = ContentScale.Crop,
65                     modifier =
Modifier.fillMaxHeight().width(130.dp)
66                 )
67
68                 Column(
69                     modifier =
Modifier.weight(1f).padding(16.dp)
70                 ) {
71                     Text(
72                         text = product.name ?: "Unknown
Product",
73                         style =
MaterialTheme.typography.titleMedium,
74                         fontWeight = FontWeight.Bold,
75                         maxLines = 2,
76                         overflow = TextOverflow.Ellipsis
77                     )
78                     Spacer(modifier = Modifier.height(4.dp))
79                     Text(
80                         text = product.productType ?:
"Unknown Type",
81                         style =
MaterialTheme.typography.bodySmall,
82                         color =
MaterialTheme.colorScheme.onSurface.copy(alpha = 0.7f)
83                     )
84                     Spacer(modifier = Modifier.height(8.dp))
85                     Text(
86                         text = formatPrice(product.price),

```

```

87         style =
MaterialTheme.typography.bodyLarge,
88         fontWeight = FontWeight.Bold,
89         color =
MaterialTheme.colorScheme.primary
90     )
91     Spacer(modifier = Modifier.height(8.dp))
92     Text(
93         text = "Deskripsi:
${product.description ?: ""}",
94         style =
MaterialTheme.typography.bodySmall,
95         maxLines = 2,
96         overflow = TextOverflow.Ellipsis,
97         color =
MaterialTheme.colorScheme.onSurface.copy(alpha = 0.8f)
98     )
99     Spacer(modifier = Modifier.weight(1f))
100
101     Row(
102         modifier = Modifier.fillMaxWidth(),
103         horizontalArrangement =
Arrangement.spacedBy(8.dp)
104     ) {
105         Button(
106             onClick = {
onVisitClick(product.imageLink) },
107             enabled = isOnline,
108             modifier = Modifier.weight(1f),
109             shape =
RoundedCornerShape(12.dp),
110             colors =
ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.primary)
111         ) { Text("Kunjungi", fontSize =
12.sp) }
112
113         Button(
114             onClick = {
onDetailClick(product.id) },
115             enabled = isOnline,
116             modifier = Modifier.weight(1f),
117             shape =
RoundedCornerShape(12.dp),
118             colors =
ButtonDefaults.buttonColors(
119                 containerColor =
MaterialTheme.colorScheme.secondary,
120                 contentColor = Color.Black
121             )
122         ) { Text("Detail", fontSize = 12.sp)
}

```

```

123         }
124     }
125 }
126
127     IconButton(
128         onClick = { onToggleFavorite(product.id) },
129         modifier =
130     Modifier.align(Alignment.TopEnd).padding(8.dp)
131     ) {
132         Icon(
133             imageVector = if (product.isFavorite)
134             Icons.Filled.Favorite else Icons.Outlined.FavoriteBorder,
135             contentDescription = "Toggle Favorite",
136             tint = if (product.isFavorite)
137             MaterialTheme.colorScheme.primary else Color.White
138         )
139     }

```

Tabel 12. Source Code ProductItem.kt Jawaban Soal 1

13. Navigation.kt

```

1 package com.example.modul5.presentation.navigation
2
3 object Navigation {
4     const val ROUTE_LIST = "makeup_list"
5     const val ARG_MAKEUP_ID = "makeupId"
6     const val ROUTE_DETAIL = "makeup_detail/{\$ARG_MAKEUP_ID}"
7
8     fun createDetailRoute(makeupId: Int): String {
9         return "makeup_detail/\$makeupId"
10    }
11 }

```

Tabel 13. Source Code Navigation.kt Jawaban Soal 1

14. ApiMakeupItem.kt

```

1 package com.example.modul5.presentation.screen
2
3 import android.util.Log
4 import androidx.compose.foundation.Image
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.foundation.shape.RoundedCornerShape
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.draw.clip
11 import androidx.compose.ui.text.style.TextOverflow
12 import androidx.compose.ui.unit.dp
13 import coil.compose.rememberAsyncImagePainter

```

```

14 import com.example.modul5.data.model.MakeupProduct
15
16 private const val TAG = "ApiMakeupItem"
17
18 @Composable
19 fun ApiMakeupItem(
20     product: MakeupProduct,
21     onVisitClick: () -> Unit,
22     onDetailClick: () -> Unit
23 ) {
24     Card(
25         shape = RoundedCornerShape(20.dp),
26         modifier = Modifier.fillMaxWidth(),
27         colors = CardDefaults.cardColors(
28             containerColor =
29                 MaterialTheme.colorScheme.surface
30         ),
31         elevation =
32             CardDefaults.cardElevation(defaultElevation = 8.dp)
33     ) {
34         Row(
35             modifier = Modifier
36                 .padding(16.dp)
37                 .fillMaxWidth()
38         ) {
39             Image(
40                 painter =
41                     rememberAsyncImagePainter(product.imageLink),
42                 contentDescription = product.name ?: "",
43                 modifier = Modifier
44                     .size(100.dp, 150.dp)
45                     .clip(RoundedCornerShape(16.dp))
46             )
47             Spacer(modifier = Modifier.width(16.dp))
48             Column(modifier = Modifier.weight(1f)) {
49                 Text(
50                     text = product.name ?: "Tanpa Nama",
51                     style =
52                         MaterialTheme.typography.titleMedium,
53                     color =
54                         MaterialTheme.colorScheme.primary,
55                     maxLines = 2
56                 )
57                 Spacer(modifier = Modifier.height(4.dp))
58                 Text(
59                     text = "${product.productType ?: "?"} |
60                     Rp${product.price ?: "-"}",

```

59	style =
60	MaterialTheme.typography.bodySmall,
61	color =
62	MaterialTheme.colorScheme.primary
63	)
64	Spacer(modifier = Modifier.height(8.dp))
65	Text(
66	text = "Deskripsi:
67	`\${product.description ? : "-"}`,
68	style =
69	MaterialTheme.typography.bodySmall,
70	color =
71	MaterialTheme.colorScheme.onSurface,
72	maxLines = 4,
73	overflow = TextOverflow.Ellipsis
74	)
75	Spacer(modifier = Modifier.height(12.dp))
76	Row(
77	modifier = Modifier.fillMaxWidth(),
78	horizontalArrangement =
79	Arrangement.SpaceEvenly
80	) {
81	Button(
82	onClick = {
83	Log.d(TAG, "Visit (API
84	Clicked")
85	onVisitClick()
86	},
87	colors =
88	ButtonDefaults.buttonColors(
89	containerColor =
90	MaterialTheme.colorScheme.primary,
91	contentColor =
92	MaterialTheme.colorScheme.onPrimary
93	),
94	shape = RoundedCornerShape(12.dp),
95	elevation =
96	ButtonDefaults.buttonElevation(6.dp)
97	) {
	Text("Kunjungi", style =
	MaterialTheme.typography.bodyMedium)
	}
	Button(
	onClick = {
	Log.d(TAG, "Detail (API
	Clicked")
	onDetailClick()

98	},
99	colors =
	ButtonDefaults.buttonColors(
100	containerColor =
	MaterialTheme.colorScheme.secondary,
101	contentColor =
	MaterialTheme.colorScheme.onSecondary
102	),
103	shape = RoundedCornerShape(12.dp),
104	elevation =
	ButtonDefaults.buttonElevation(6.dp)
105	) {
106	Text("Detail", style =
	MaterialTheme.typography.bodyMedium)
107	}
108	}
109	}
110	}
111	}
112	}

Tabel 14. Source Code ApiMakeupItem.kt Jawaban Soal 1

15. MakeupItem.kt

1	package com.example.modul5.presentation.screen
2	
3	import android.util.Log
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Modifier
9	import androidx.compose.ui.draw.clip
10	import androidx.compose.ui.layout.ContentScale
11	import androidx.compose.ui.platform.LocalContext
12	import androidx.compose.ui.text.style.TextOverflow
13	import androidx.compose.ui.unit.dp
14	import coil.compose.AsyncImage
15	import coil.request.ImageRequest
16	import com.example.modul5.data.model.MakeupProduct
17	
18	private const val TAG = "MakeupItem"
19	
20	@Composable
21	fun MakeupItem(
22	product: MakeupProduct,
23	onVisitClick: (String) -> Unit,
24	onDetailClick: (Int) -> Unit
25	) {
26	Card(
27	shape = RoundedCornerShape(20.dp),
28	modifier = Modifier.fillMaxWidth(),

```

29         colors = CardDefaults.cardColors(
30             containerColor =
MaterialTheme.colorScheme.surface
31         ),
32         elevation =
CardDefaults.cardElevation(defaultElevation = 8.dp)
33     ) {
34         Row(
35             modifier = Modifier
36                 .padding(16.dp)
37                 .fillMaxWidth()
38         ) {
39             AsyncImage(
40                 model =
ImageRequest.Builder(LocalContext.current)
41                     .data(product.imageUrl)
42                     .crossfade(true)

43             .error(android.R.drawable.ic_menu_report_image)
44                 .build(),
45                 contentDescription = product.name,
46                 contentScale = ContentScale.Crop,
47                 modifier = Modifier
48                     .size(100.dp, 150.dp)
49                     .clip(RoundedCornerShape(16.dp))
50             )
51
52             Spacer(modifier = Modifier.width(16.dp))
53
54             Column(modifier = Modifier.weight(1f)) {
55                 Text(
56                     text = product.name ?: "Tanpa Nama",
57                     style =
MaterialTheme.typography.titleMedium,
58                     color =
MaterialTheme.colorScheme.primary,
59                     maxLines = 2
60                 )
61
62                 Spacer(modifier = Modifier.height(4.dp))
63
64                 Text(
65                     text =
"${product.brand?.replaceFirstChar { it.uppercase() }} |
${product.productType}",
66                     style =
MaterialTheme.typography.bodySmall,
67                     color =
MaterialTheme.colorScheme.primary
68                 )
69
69                 Spacer(modifier = Modifier.height(8.dp))

```

```

70
71         Text(
72             text = "Deskripsi:
73     ${product.description ?: "-"}",
74             style =
75     MaterialTheme.typography.bodySmall,
76             color =
77     MaterialTheme.colorScheme.onSurface,
78             maxLines = 4,
79             overflow = TextOverflow.Ellipsis
80         )
81
82         Spacer(modifier = Modifier.height(12.dp))
83
84         Row(
85             modifier = Modifier.fillMaxWidth(),
86             horizontalArrangement =
87     Arrangement.SpaceEvenly
88         ) {
89             Button(
90                 onClick = {
91                     Log.d(TAG, "Tombol Visit
92     ditekan: ${product.imageLink}")
93                     onVisitClick(product.imageLink
94     ?: "#")
95                 },
96                 colors =
97     ButtonDefaults.buttonColors(
98                 containerColor =
99     MaterialTheme.colorScheme.primary,
100                contentColor =
101     MaterialTheme.colorScheme.onPrimary
102            ),
103                shape = RoundedCornerShape(12.dp),
104                elevation =
105     ButtonDefaults.buttonElevation(6.dp)
106            ) {
107                Text("Kunjungi", style =
108     MaterialTheme.typography.bodyMedium)
109            }
110
111            Button(
112                onClick = {
113                    Log.d(TAG, "Tombol Detail
114     ditekan: ${product.id}")
115                    onDetailClick(product.id)
116                },
117                colors =
118     ButtonDefaults.buttonColors(
119                containerColor =
120     MaterialTheme.colorScheme.secondary,

```

108	contentColor =
109	MaterialTheme.colorScheme.onSecondary
110	),
111	shape = RoundedCornerShape(12.dp),
112	elevation =
113	ButtonDefaults.buttonElevation(6.dp)
114	) {
115	Text("Detail", style =
116	MaterialTheme.typography.bodyMedium)
117	}
118	}
119	}

Tabel 15. Source Code MakeupItem.kt Jawaban Soal 1

16. MakeupListScreen.kt

1	package com.example.modul5.presentation.screen
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.widget.Toast
6	import androidx.compose.foundation.background
7	import androidx.compose.foundation.layout.*
8	import androidx.compose.foundation.lazy.LazyColumn
9	import androidx.compose.foundation.lazy.items
10	import androidx.compose.material.icons.Icons
11	import androidx.compose.material.icons.filled.WifiOff
12	import androidx.compose.material3.*
13	import androidx.compose.runtime.*
14	import androidx.compose.ui.Alignment
15	import androidx.compose.ui.Modifier
16	import androidx.compose.ui.platform.LocalContext
17	import androidx.compose.ui.text.font.FontWeight
18	import androidx.compose.ui.text.style.TextAlign
19	import androidx.compose.ui.unit.dp
20	import
21	androidx.lifecycle.compose.collectAsStateWithLifecycle
22	import androidx.lifecycle.viewmodel.compose.viewModel
23	import androidx.navigation.NavController
24	import
25	com.example.modul5.presentation.components.ProductItem
26	import
27	com.example.modul5.presentation.viewmodel.MakeupViewModel
	com.example.modul5.presentation.viewmodel.MakeupViewModelFa
	ctory

```

28 import kotlinx.coroutines.flow.collectLatest
29
30 @OptIn(ExperimentalMaterial3Api::class)
31 @Composable
32 fun MakeupListScreen(navController: NavController) {
33     val context = LocalContext.current
34     val repository = (context.applicationContext as
35 MakeupApp).repository
36     val viewModel: MakeupViewModel = viewModel(factory =
37 MakeupViewModelFactory(repository))
38
39     val connectivityObserver = (context.applicationContext
40 as MakeupApp).connectivityObserver
41     val isOnline by
42 connectivityObserver.observe().collectAsState(initial =
43 false)
44
45     val uiState by
46 viewModel.uiState.collectAsStateWithLifecycle()
47     var selectedTabIndex by remember { mutableStateOf(0) }
48     val tabs = listOf("Home", "Favorit")
49     val localContext = LocalContext.current
50
51     LaunchedEffect(key1 = true) {
52         viewModel.eventFlow.collectLatest { event ->
53             when (event) {
54                 is MakeupViewModel.UiEvent.NavigateToDetail
55                 ->
56                 navController.navigate(Navigation.createDetailRoute(event.m
57 akeupId))
58
59                 is MakeupViewModel.UiEvent.OpenWebUrl -> {
60                     val intent = Intent(Intent.ACTION_VIEW,
61 Uri.parse(event.url))
62                     try {
63                         localContext.startActivity(intent) }
64                     catch (e: Exception) {
65                         Toast.makeText(localContext, "Tidak dapat membuka link",
66 Toast.LENGTH_SHORT).show() }
67                     }
68             }
69         }
70
71     Scaffold(
72         containerColor =
73 MaterialTheme.colorScheme.background,
74         topBar = {
75             TopAppBar(
76                 title = { Text("Makeup App", fontWeight =
77 FontWeight.Bold, color =
78 MaterialTheme.colorScheme.onBackground) },

```

```

63         colors =
TopAppBarDefaults.topAppBarColors(containerColor =
MaterialTheme.colorScheme.background)
64     )
65     }
66     ) { paddingValues ->
67         Column(modifier = Modifier.padding(paddingValues))
{
68             TabRow(
69                 selectedTabIndex = selectedTabIndex,
70                 containerColor =
MaterialTheme.colorScheme.background,
71                 contentColor =
MaterialTheme.colorScheme.primary
72             ) {
73                 tabs.forEachIndexed { index, title ->
74                     Tab(
75                         selected = selectedTabIndex ==
index,
76                         onClick = { selectedTabIndex =
index },
77                         text = { Text(text = title,
fontWeight = FontWeight.Bold) }
78                     )
79                 }
80             }
81
82             if (!isOnline) {
83                 Row(
84                     modifier = Modifier
85                         .fillMaxWidth()

86                 .background(MaterialTheme.colorScheme.errorContainer)
87                     .padding(horizontal = 16.dp,
vertical = 8.dp),
88                     verticalAlignment =
Alignment.CenterVertically,
89                     horizontalArrangement =
Arrangement.Center
90                 ) {
91                     Icon(
92                         Icons.Default.WifiOff,
93                         contentDescription = "Offline",
94                         tint =
MaterialTheme.colorScheme.onErrorContainer
95                     )
96                     Spacer(modifier = Modifier.width(8.dp))
97                     Text(
98                         text = "Anda sedang offline",
99                         color =
MaterialTheme.colorScheme.onErrorContainer,
100                        fontWeight = FontWeight.Medium

```

```

101         )
102     }
103 }
104
105     when (selectedTabIndex) {
106         0 -> ProductList(
107             products = uiState.products,
108             isLoading = uiState.isLoading,
109             error = uiState.error,
110             viewModel = viewModel,
111             isOnline = isOnline // Kirim status
112         koneksi
113         )
114         1 -> ProductList(
115             products = uiState.favorites,
116             isLoading = false,
117             error = null,
118             viewModel = viewModel,
119             isOnline = isOnline, // Kirim status
120         koneksi
121         emptyMessage = "Anda belum memiliki
122         produk favorit."
123     )
124 }
125
126 @Composable
127 fun ProductList(
128     products:
129     List<com.example.modul5.data.model.MakeupProduct>,
130     isLoading: Boolean,
131     error: String?,
132     viewModel: MakeupViewModel,
133     isOnline: Boolean,
134     emptyMessage: String = "Tidak ada produk yang
135     ditemukan."
136 ) {
137     Box(modifier = Modifier.fillMaxSize()) {
138         if (isLoading && products.isEmpty()) {
139             CircularProgressIndicator(modifier =
140             Modifier.align(Alignment.Center))
141         } else if (error != null) {
142             Text(text = error, modifier =
143             Modifier.align(Alignment.Center).padding(16.dp), textAlign
144             = TextAlign.Center)
145         } else if (products.isEmpty()) {
146             Text(text = emptyMessage, modifier =
147             Modifier.align(Alignment.Center).padding(16.dp), textAlign
148             = TextAlign.Center, color =
149             MaterialTheme.colorScheme.onSurface.copy(alpha = 0.7f))
150         }
151     }
152 }

```

142	} else {
143	LazyColumn(
144	contentPadding = PaddingValues(16.dp),
145	verticalArrangement =
	Arrangement.spacedBy(16.dp)
146	) {
147	items(products, key = { it.id }) { product
148	->
148	ProductItem(
149	product = product,
150	onVisitClick = {
	viewModel.onVisitClicked(product.imageLink) },
151	onDetailClick = {
	viewModel.onDetailClicked(product.id) },
152	onToggleFavorite = {
	viewModel.toggleFavorite(product.id) },
153	isOnline = isOnline
154	)
155	}
156	}
157	}
158	}
159	}

Tabel 16. Source Code MakeupListScreen.kt Jawaban Soal 1

17. MakeupDetailScreen.kt

1	package com.example.modul5.presentation.screen
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.rememberScrollState
5	import androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.foundation.verticalScroll
7	import androidx.compose.material.icons.Icons
8	import androidx.compose.material.icons.automirrored.filled.ArrowBack
9	import androidx.compose.material.icons.filled.Favorite
10	import androidx.compose.material.icons.outlined.FavoriteBorder
	import androidx.compose.material3.*
11	import androidx.compose.runtime.Composable
12	import androidx.compose.runtime.LaunchedEffect
13	import androidx.compose.runtime.getValue
14	import androidx.compose.ui.Alignment
15	import androidx.compose.ui.Modifier
16	import androidx.compose.ui.draw.clip
17	import androidx.compose.ui.graphics.Color
18	import androidx.compose.ui.layout.ContentScale
19	import androidx.compose.ui.platform.LocalContext
20	import androidx.compose.ui.res.painterResource
21	import androidx.compose.ui.text.font.FontWeight

```

22 import androidx.compose.ui.text.style.TextAlign
23 import androidx.compose.ui.unit.dp
24 import androidx.compose.ui.unit.sp
25 import
26 androidx.lifecycle.compose.collectAsStateWithLifecycle
27 import androidx.lifecycle.viewmodel.compose.viewModel
28 import androidx.navigation.NavController
29 import coil.compose.AsyncImage
30 import coil.request.ImageRequest
31 import com.example.modul5.MakeupApp
32 import com.example.modul5.R
33 import com.example.modul5.presentation.viewmodel.MakeupViewModel
34 import com.example.modul5.presentation.viewmodel.MakeupViewModelFactory
35
36 @OptIn(ExperimentalMaterial3Api::class)
37 @Composable
38 fun MakeupDetailScreen(
39     navController: NavController,
40     makeupId: Int
41 ) {
42     val context = LocalContext.current
43     val repository = (context.applicationContext as
44 MakeupApp).repository
45     val viewModel: MakeupViewModel = viewModel(factory =
46 MakeupViewModelFactory(repository))
47
48     LaunchedEffect(key1 = makeupId) {
49         viewModel.getProductDetails(makeupId)
50     }
51
52     val product by
53     viewModel.detailProduct.collectAsStateWithLifecycle()
54
55     Scaffold(
56         containerColor =
57         MaterialTheme.colorScheme.background,
58         topBar = {
59             TopAppBar(
60                 title = { },
61                 navigationIcon = {
62                     IconButton(onClick = {
63                         navController.popBackStack() }) {
64                         Icon(Icons.AutoMirrored.Filled.ArrowBack, "Kembali", tint =
65                         MaterialTheme.colorScheme.primary)
66                     }
67                 },
68                 actions = {

```

```

63         product?.let { p ->
64             IconButton(onClick = {
65                 viewModel.toggleFavorite(p.id) }) {
66                 Icon(
67                     imageVector = if
68                     (p.isFavorite) Icons.Filled.Favorite else
69                     Icons.Outlined.FavoriteBorder,
70                     contentDescription = "Toggle
71                     Favorite",
72                     tint = if (p.isFavorite)
73                     MaterialTheme.colorScheme.primary else Color.Gray,
74                     modifier =
75                     Modifier.size(28.dp)
76                 )
77             }
78         },
79         colors =
80         TopAppBarDefaults.topAppBarColors(containerColor =
81         Color.Transparent)
82     )
83     ) { paddingValues ->
84         Box(
85             modifier = Modifier
86             .fillMaxSize()
87             .padding(paddingValues)
88         ) {
89             if (product == null) {
90                 CircularProgressIndicator(modifier =
91                 Modifier.align(Alignment.Center))
92             } else {
93                 val p = product!!
94                 Column(
95                     modifier = Modifier
96                     .fillMaxSize()
97                     .verticalScroll(rememberScrollState()),
98                     horizontalAlignment =
99                     Alignment.CenterHorizontally
100                 ) {
101                     AsyncImage(
102                         model =
103                         ImageRequest.Builder(LocalContext.current)
104                         .data(p.directImageLink)
105                         .crossfade(true)
106                         .build(),
107                         placeholder = painterResource(id =
108                         R.drawable.ic_launcher_foreground),
109                         error = painterResource(id =
110                         R.drawable.ic_launcher_foreground),
111                         contentDescription = p.name,

```

```

101         contentScale = ContentScale.Fit,
102         modifier = Modifier
103             .fillMaxWidth()
104             .height(300.dp)
105             .padding(horizontal = 24.dp)
106             .clip(RoundedCornerShape(20.dp))
107     )
108     Spacer(modifier =
109 Modifier.height(24.dp))
110     Column(modifier =
111 Modifier.padding(horizontal = 24.dp)) {
112         Text(
113             text = p.name ?: "Nama Produk
114 Tidak Tersedia",
115             style =
116 MaterialTheme.typography.headlineMedium,
117             textAlign = TextAlign.Center,
118             fontWeight = FontWeight.Bold,
119             color =
120 MaterialTheme.colorScheme.primary,
121             modifier =
122 Modifier.fillMaxWidth()
123         )
124         Spacer(modifier =
125 Modifier.height(8.dp))
126         Text(
127             text = p.productType ?: "Tipe
128 Produk",
129             style =
130 MaterialTheme.typography.titleMedium,
131             color =
132 MaterialTheme.colorScheme.onSurface.copy(alpha = 0.7f),
133             textAlign = TextAlign.Center,
134             modifier =
135 Modifier.fillMaxWidth()
136         )
137         Spacer(modifier =
138 Modifier.height(32.dp))
139         Text(
140             text = "Deskripsi",
141             style =
142 MaterialTheme.typography.titleLarge,
143             fontWeight = FontWeight.Bold
144         )
145         Spacer(modifier =
146 Modifier.height(8.dp))
147         Text(
148             text = p.description ?: "Tidak
149 ada deskripsi yang tersedia untuk produk ini.",

```

```

138         style =
MaterialTheme.typography.bodyLarge,
139         lineHeight = 24.sp,
140         textAlign = TextAlign.Justify
141     )
142     }
143
144     Spacer(modifier =
Modifier.height(32.dp))
145
146     Button(
147         onClick = {
navController.popBackStack() },
148         modifier = Modifier
149             .fillMaxWidth()
150             .padding(horizontal = 24.dp)
151             .height(50.dp),
152         shape = RoundedCornerShape(16.dp),
153         colors =
ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.primary)
154     ) {
155         Text("Kembali", fontSize = 16.sp)
156     }
157     Spacer(modifier =
Modifier.height(24.dp))
158     }
159     }
160 }
161 }
162 }

```

Tabel 17. Source Code MakeupDetailScreen.kt Jawaban Soal 1

18. MakeupViewModel.kt

```

1 package com.example.modul5.presentation.viewmodel
2
3 import android.util.Log
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.viewModelScope
6 import com.example.modul5.data.model.MakeupProduct
7 import com.example.modul5.data.repository.MakeupRepository
8 import kotlinx.coroutines.flow.*
9 import kotlinx.coroutines.launch
10
11 data class MakeupListUiState(
12     val isLoading: Boolean = true,
13     val products: List<MakeupProduct> = emptyList(),
14     val favorites: List<MakeupProduct> = emptyList(),
15     val error: String? = null
16 )
17

```

```

18 class MakeupViewModel(private val repository:
    MakeupRepository) : ViewModel() {
19
20     private val TAG = "MakeupViewModel"
21
22     private val _uiState =
    MutableStateFlow(MakeupListUiState())
23     val uiState: StateFlow<MakeupListUiState> =
    _uiState.asStateFlow()
24
25     private val _detailProduct =
    MutableStateFlow<MakeupProduct?>(null)
26     val detailProduct: StateFlow<MakeupProduct?> =
    _detailProduct.asStateFlow()
27
28     private val _eventFlow = MutableSharedFlow<UiEvent>()
29     val eventFlow: SharedFlow<UiEvent> =
    _eventFlow.asSharedFlow()
30
31     init {
32         observeProductsFromCache()
33         observeFavorites()
34         refreshProducts()
35     }
36
37     private fun refreshProducts() {
38         viewModelScope.launch {
39             _uiState.update { it.copy(isLoading = true) }
40             repository.fetchAndCacheProducts()
41             .catch { e ->
42                 _uiState.update { it.copy(isLoading =
    false, error = "Gagal memuat data: ${e.message}") }
43             }
44             .launchIn(viewModelScope)
45         }
46     }
47
48     private fun observeProductsFromCache() {
49         repository.getProductsFromCache()
50             .onEach { products ->
51                 Log.d(TAG, "Data Home diperbarui dari cache:
    ${products.size} item.")
52                 _uiState.update { it.copy(isLoading = false,
    products = products) }
53             }
54             .launchIn(viewModelScope)
55     }
56
57     private fun observeFavorites() {
58         repository.getFavoriteProducts()
59             .onEach { favoriteProducts ->

```

```

60         Log.d(TAG, "Data Favorit diperbarui dari
cache: ${favoriteProducts.size} item.")
61         _uiState.update { it.copy(favorites =
favoriteProducts) }
62     }
63     .launchIn(viewModelScope)
64 }
65
66 fun getProductDetails(id: Int) {
67     viewModelScope.launch {
68         Log.d(TAG, "Mengambil detail untuk produk ID:
$id")
69         repository.getProductById(id).collect { product
->
70             _detailProduct.value = product
71         }
72     }
73 }
74
75 fun toggleFavorite(id: Int) {
76     viewModelScope.launch {
77         Log.d(TAG, "Tombol Favorit ditekan untuk ID:
$id")
78         val productToToggle =
_detailProduct.value?.takeIf { it.id == id }
79         ?: _uiState.value.products.find { it.id ==
id }
80         ?: _uiState.value.favorites.find { it.id ==
id }
81
82         productToToggle?.let {
83             repository.toggleFavoriteStatus(id,
!it.isFavorite)
84         }
85     }
86 }
87
88 fun onDetailClicked(id: Int) {
89     Log.d(TAG, "Tombol Detail ditekan untuk ID: $id")
90     val selectedProduct = _uiState.value.products.find {
it.id == id } ?: _uiState.value.favorites.find { it.id == id
}
91     Log.d(TAG, "Data yang dipilih:
${selectedProduct?.name}")
92     viewModelScope.launch {
93         _eventFlow.emit(UiEvent.NavigateToDetail(id))
94     }
95 }
96
97 fun onVisitClicked(url: String?) {
98     Log.d(TAG, "Tombol Kunjungi ditekan. URL: $url")
99     if (!url.isNullOrEmpty()) {

```

100	viewModelScope.launch {
101	_eventFlow.emit(UiEvent.OpenWebUrl(url))
102	}
103	}
104	}
105	
106	sealed class UiEvent {
107	data class NavigateToDetail(val makeupId: Int) :
108	UiEvent()
109	data class OpenWebUrl(val url: String) : UiEvent()
110	}

Tabel 18. Source Code MakeupViewModel.kt Jawaban Soal 1

19. MakeupViewModelFactory.kt

1	package com.example.modul5.presentation.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.modul5.data.repository.MakeupRepository
6	
7	class MakeupViewModelFactory(private val repository:
8	MakeupRepository) : ViewModelProvider.Factory {
9	override fun <T : ViewModel> create(modelClass:
10	Class<T>): T {
11	if
12	(modelClass.isAssignableFrom(MakeupViewModel::class.java)) {
13	@Suppress("UNCHECKED_CAST")
14	return MakeupViewModel(repository) as T
15	}
	throw IllegalArgumentException("Unknown ViewModel
	class")
	}
	}

Tabel 19. Source Code MakeupViewModelFactory.kt Jawaban Soal 1

20. MainActivity.kt

1	package com.example.modul5.presentation
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.material3.MaterialTheme
9	import androidx.compose.material3.Surface
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.Modifier
12	import androidx.navigation.NavHostController
13	import androidx.navigation.NavType

```

14 import androidx.navigation.compose.NavHost
15 import androidx.navigation.compose.composable
16 import androidx.navigation.compose.rememberNavController
17 import androidx.navigation.navArgument
18 import com.example.modul5.presentation.navigation.Navigation
19 import
20 com.example.modul5.presentation.screen.MakeupDetailScreen
21 import
22 com.example.modul5.presentation.screen.MakeupListScreen
23 import com.example.modul5.ui.theme.Modul5Theme
24
25 class MainActivity : ComponentActivity() {
26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         enableEdgeToEdge()
29         setContent {
30             Modul5Theme {
31                 Surface(
32                     modifier = Modifier.fillMaxSize(),
33                     color =
34                     MaterialTheme.colorScheme.background
35                 ) {
36                     MainApp()
37                 }
38             }
39         }
40     }
41
42 @Composable
43 fun MainApp() {
44     val navController: NavController =
45     rememberNavController()
46
47     NavHost(
48         navController = navController,
49         startDestination = Navigation.ROUTE_LIST,
50     ) {
51         composable(Navigation.ROUTE_LIST) {
52             MakeupListScreen(navController = navController)
53         }
54         composable(
55             route = Navigation.ROUTE_DETAIL,
56             arguments =
57             listOf(navArgument(Navigation.ARG_MAKEUP_ID) {
58                 type = NavType.IntType
59             })
60         ) { backStackEntry ->
61             val makeupId =
62             backStackEntry.arguments!!.getInt(Navigation.ARG_MAKEUP_ID)
63             MakeupDetailScreen(navController =
64             navController, makeupId = makeupId)
65         }
66     }
67 }

```

59	}
60	}
61	}

Tabel 20. Source Code MainActivity.kt Jawaban Soal 1

21. Color.kt

1	package com.example.modul5.ui.theme
2	
3	import androidx.compose.ui.graphics.Color
4	
5	val SoftPink = Color(0xFFFFC1E3)
6	val LightLilac = Color(0xFFE6DAF5)
7	val BlushPink = Color(0xFFFFD6E8)
8	
9	val Rose = Color(0xFFCB0045)
10	val Plum = Color(0xFF8E24AA)
11	val Mauve = Color(0xFFCE93D8)

Tabel 21. Source Code Color.kt Jawaban Soal 1

22. Theme.kt

1	package com.example.modul5.ui.theme
2	
3	import android.os.Build
4	import androidx.compose.foundation.isSystemInDarkTheme
5	import androidx.compose.material3.*
6	import androidx.compose.runtime.Composable
7	import androidx.compose.ui.graphics.Color
8	import androidx.compose.ui.platform.LocalContext
9	
10	private val DarkColorScheme = darkColorScheme(
11	primary = Mauve,
12	secondary = LightLilac,
13	tertiary = BlushPink,
14	background = Color(0xFF2A2A2E),
15	surface = Color(0xFF2A2A2E),
16	onPrimary = Color.White,
17	onSecondary = Color.Black,
18	onTertiary = Color.White,
19	onBackground = Color(0xFFEAEAEA),
20	onSurface = Color(0xFFEAEAEA)
21	)
22	
23	private val LightColorScheme = lightColorScheme(
24	primary = Rose,
25	secondary = SoftPink,
26	tertiary = LightLilac,
27	background = Color(0xFFFFF0F5),
28	surface = Color(0xFFFFF5F7),
29	onPrimary = Color.White,
30	onSecondary = Rose,

```

31     onTertiary = Color.Black,
32     onBackground = Color(0xFF333333),
33     onSurface = Color(0xFF333333)
34 )
35
36 @Composable
37 fun Modul5Theme(
38     darkTheme: Boolean = isSystemInDarkTheme(),
39     dynamicColor: Boolean = false,
40     content: @Composable () -> Unit
41 ) {
42     val colorScheme = when {
43         dynamicColor && Build.VERSION.SDK_INT >=
44             Build.VERSION_CODES.S -> {
45                 val context = LocalContext.current
46                 if (darkTheme) dynamicDarkColorScheme(context)
47             else dynamicLightColorScheme(context)
48         }
49         darkTheme -> DarkColorScheme
50         else -> LightColorScheme
51     }
52
53     MaterialTheme(
54         colorScheme = colorScheme,
55         typography = Typography,
56         content = content
57     )
58 }

```

Tabel 22. Source Code Theme.kt Jawaban Soal 1

23. Type.kt

```

1 package com.example.modul5.ui.theme
2
3 import androidx.compose.material3.Typography
4 import androidx.compose.ui.text.TextStyle
5 import androidx.compose.ui.text.font.FontFamily
6 import androidx.compose.ui.text.font.FontWeight
7 import androidx.compose.ui.unit.sp
8
9 val Typography = Typography(
10     bodyLarge = TextStyle(
11         fontFamily = FontFamily.SansSerif,
12         fontWeight = FontWeight.Normal,
13         fontSize = 16.sp,
14         lineHeight = 24.sp
15     ),
16     titleLarge = TextStyle(
17         fontFamily = FontFamily.SansSerif,
18         fontWeight = FontWeight.Bold,
19         fontSize = 22.sp,
20         lineHeight = 28.sp

```

```

21     ),
22     titleMedium = TextStyle(
23         fontFamily = FontFamily.SansSerif,
24         fontWeight = FontWeight.Medium,
25         fontSize = 18.sp,
26         lineHeight = 24.sp
27     ),
28     bodyMedium = TextStyle(
29         fontFamily = FontFamily.SansSerif,
30         fontWeight = FontWeight.Normal,
31         fontSize = 14.sp,
32         lineHeight = 20.sp
33     ),
34     bodySmall = TextStyle(
35         fontFamily = FontFamily.SansSerif,
36         fontWeight = FontWeight.Normal,
37         fontSize = 12.sp,
38         lineHeight = 16.sp
39     )
40 )

```

Tabel 23. Source Code Type.kt Jawaban Soal 1

24. ConnectivityObserver.kt

```

1  package com.example.modul5.util
2
3  import android.content.Context
4  import android.net.ConnectivityManager
5  import android.net.Network
6  import android.util.Log
7  import kotlinx.coroutines.channels.awaitClose
8  import kotlinx.coroutines.flow.Flow
9  import kotlinx.coroutines.flow.callbackFlow
10 import kotlinx.coroutines.flow.distinctUntilChanged
11 import kotlinx.coroutines.launch
12
13 interface ConnectivityObserver {
14     fun observe(): Flow<Boolean>
15 }
16
17 class NetworkConnectivityObserver(
18     private val context: Context
19 ) : ConnectivityObserver {
20
21     private val connectivityManager =
22
23     context.getSystemService(Context.CONNECTIVITY_SERVICE) as
24     ConnectivityManager
25
26     override fun observe(): Flow<Boolean> {
27         return callbackFlow {
28
29

```

```

26         val callback = object :
ConnectivityManager.NetworkCallback() {
27             override fun onAvailable(network: Network) {
28                 super.onAvailable(network)
29                 Log.d("ConnectivityObserver", "Jaringan
Tersedia")
30                 launch { send(true) }
31             }
32
33             override fun onLost(network: Network) {
34                 super.onLost(network)
35                 Log.d("ConnectivityObserver", "Jaringan
Hilang")
36                 launch { send(false) }
37             }
38         }
39
40         val isConnected =
connectivityManager.activeNetwork != null
41         Log.d("ConnectivityObserver", "Status awal
koneksi: $isConnected")
42         launch { send(isConnected) }
43
44         connectivityManager.registerDefaultNetworkCallback(callback)
45         awaitClose {
46             connectivityManager.unregisterNetworkCallback(callback)
47         }
48         }.distinctUntilChanged()
49     }
50 }

```

Tabel 24. Source Code ConnectivityObserver.kt Jawaban Soal 1

25. build.gradle.kts (Module App)

```

1  plugins {
2      id("com.android.application")
3      id("org.jetbrains.kotlin.android")
4      id("org.jetbrains.kotlin.kapt")
5      id("kotlinx-serialization")
6      id("org.jetbrains.kotlin.plugin.compose")
7  }
8
9  android {
10     namespace = "com.example.modul5"
11     compileSdk = 35
12
13     defaultConfig {
14         applicationId = "com.example.modul5"
15         minSdk = 30
16         targetSdk = 35

```

```

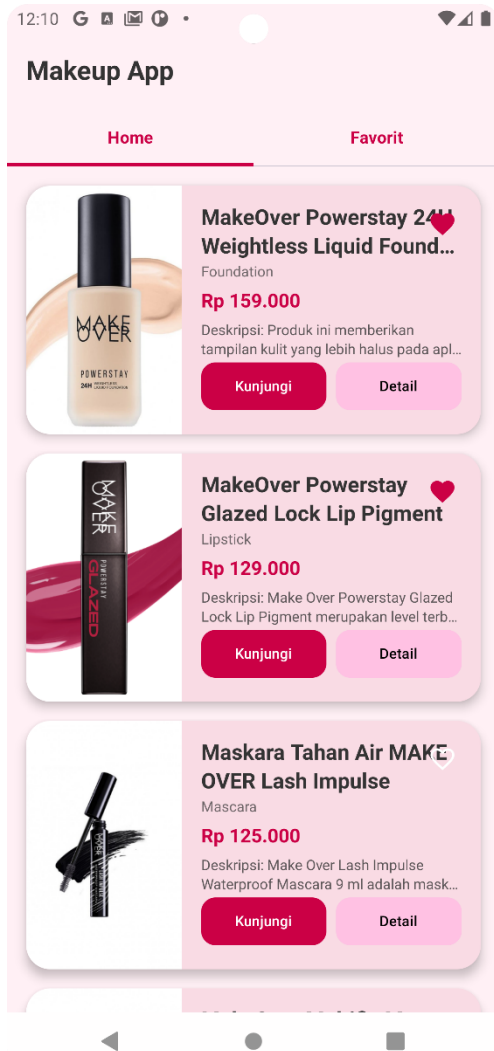
17         versionCode = 1
18         versionName = "1.0"
19
20         testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
21         vectorDrawables {
22             useSupportLibrary = true
23         }
24     }
25
26     buildTypes {
27         release {
28             isMinifyEnabled = false
29             proguardFiles(
30                 getDefaultProguardFile("proguard-android-
optimize.txt"),
31                 "proguard-rules.pro"
32             )
33         }
34     }
35
36     compileOptions {
37         sourceCompatibility = JavaVersion.VERSION_11
38         targetCompatibility = JavaVersion.VERSION_11
39     }
40
41     kotlinOptions {
42         jvmTarget = "11"
43     }
44
45     buildFeatures {
46         compose = true
47     }
48
49     packaging {
50         resources {
51             excludes += "/META-INF/{AL2.0,LGPL2.1}"
52         }
53     }
54 }
55
56 dependencies {
57     implementation("androidx.core:core-ktx:1.13.1")
58     implementation("androidx.lifecycle:lifecycle-runtime-
ktx:2.7.0")
59     implementation("androidx.activity:activity-
compose:1.9.0")
60     implementation(platform("androidx.compose:compose-
bom:2024.05.00"))
61     implementation("androidx.compose.ui:ui")
62     implementation("androidx.compose.ui:ui-graphics")
63     implementation("androidx.compose.ui:ui-tooling-preview")

```

64	implementation("androidx.compose.material3:material3")
65	implementation("androidx.compose.material:material-icons-extended")
66	implementation("androidx.navigation:navigation-compose:2.7.7")
67	implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
68	implementation("androidx.lifecycle:lifecycle-runtime-compose:2.7.0")
69	implementation("com.squareup.retrofit2:retrofit:2.9.0")
70	implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter:1.0.0")
71	implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.3")
72	implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")
73	implementation("io.coil-kt:coil-compose:2.6.0")
74	implementation("androidx.room:room-runtime:2.6.1")
75	kapt("androidx.room:room-compiler:2.6.1")
76	implementation("androidx.room:room-ktx:2.6.1")
77	implementation("androidx.datastore:datastore-preferences:1.0.0")
78	implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
79	testImplementation("junit:junit:4.13.2")
80	androidTestImplementation("androidx.test.ext:junit:1.1.5")
81	androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
82	androidTestImplementation(platform("androidx.compose:compose-bom:2024.05.00"))
83	androidTestImplementation("androidx.compose.ui:ui-test-junit4")
84	debugImplementation("androidx.compose.ui:ui-tooling")
85	debugImplementation("androidx.compose.ui:ui-test-manifest")
86	}

Tabel 25. Source Code build.gradle.kts (Module App) Jawaban Soal 1

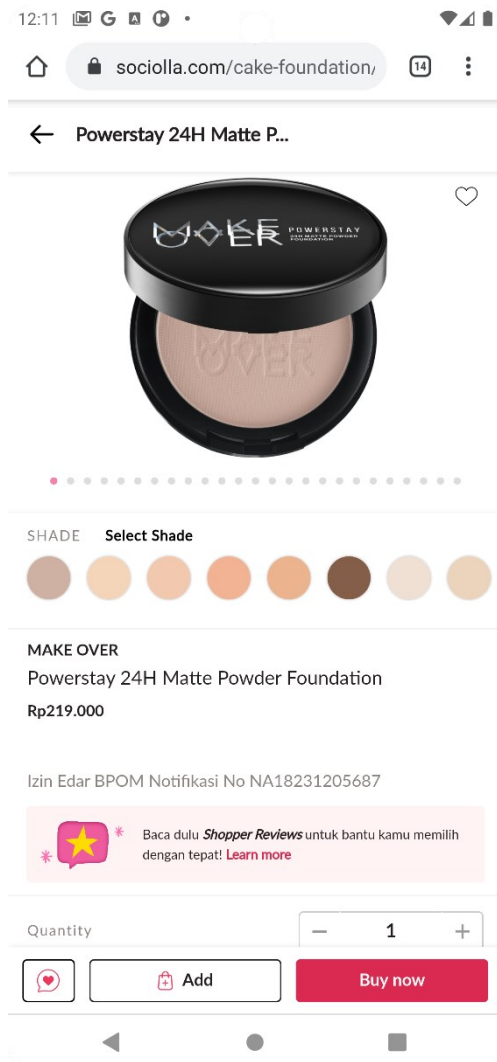
B. Output Program



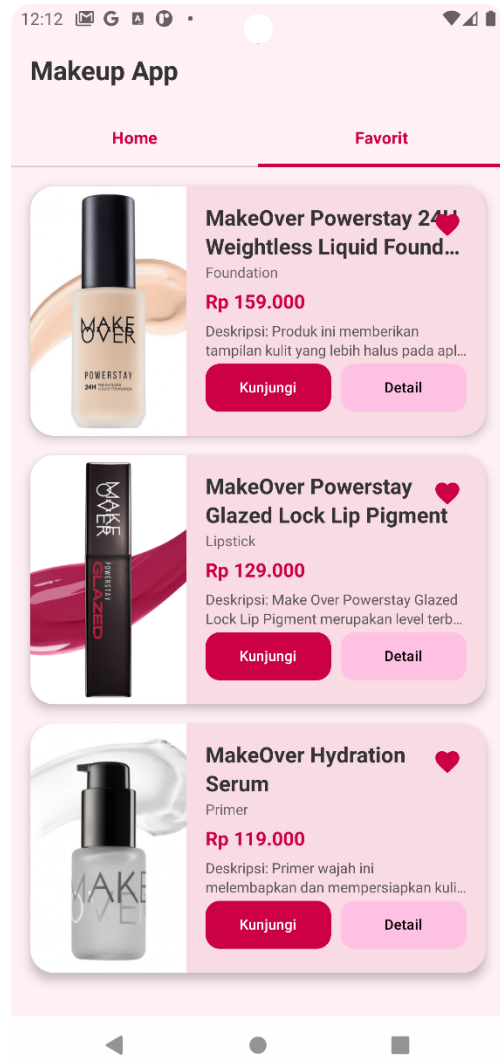
Gambar 1. Screenshot Hasil Tampilan List Jawaban Soal 1



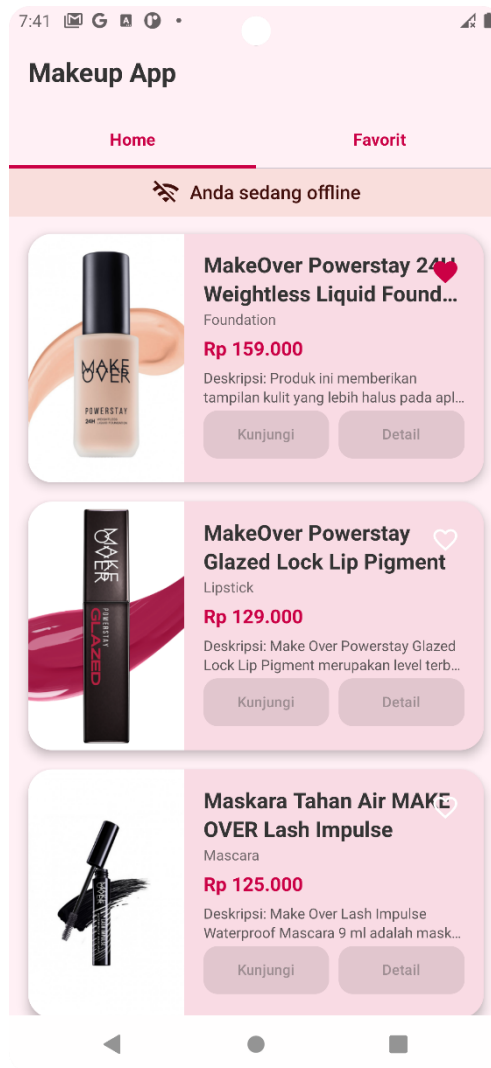
Gambar 2. Screenshot Hasil Tampilan Detail Jawaban Soal 1



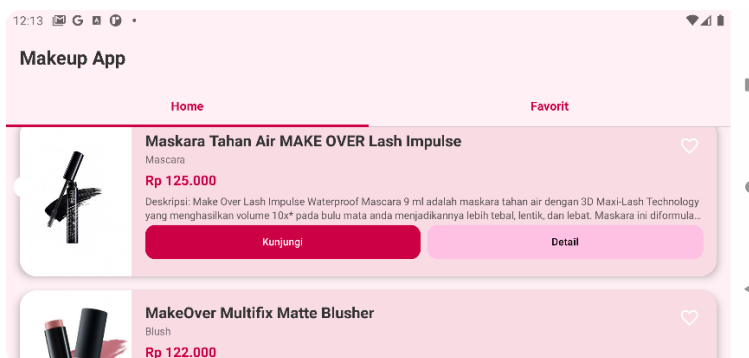
Gambar 3. Screenshot Hasil Tampilan Kunjungi Jawaban Soal 1



Gambar 4. Screenshot Hasil Tampilan Favorit Jawaban Soal 1



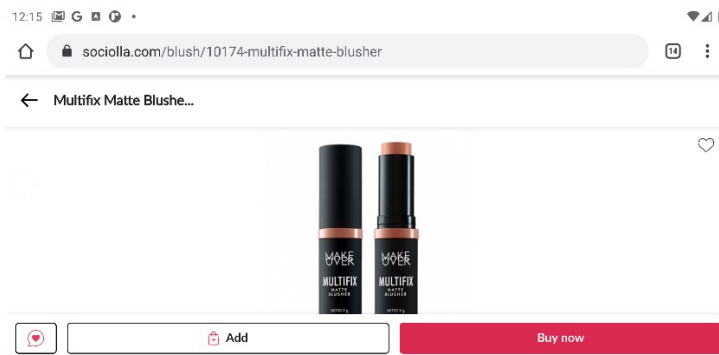
Gambar 5. Screenshot Hasil Tampilan Tanpa Koneksi Jawaban Soal



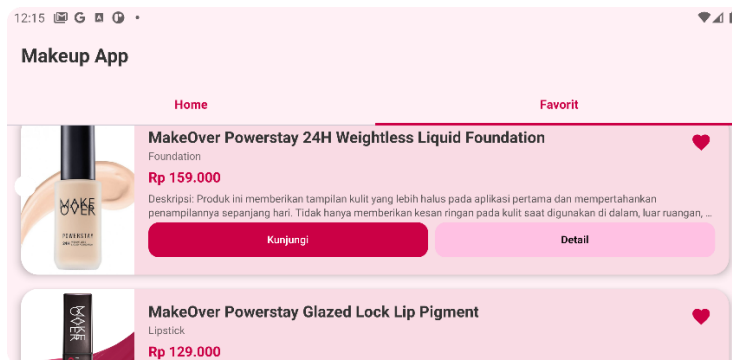
Gambar 6. Screenshot Hasil Tampilan List (landscape) Jawaban Soal 1



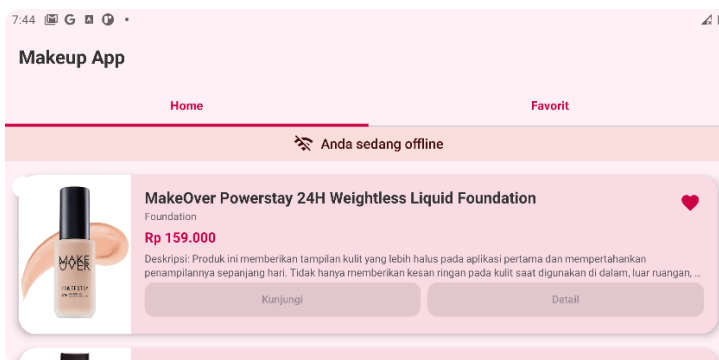
Gambar 7. Screenshot Hasil Tampilan Detail (landscape) Jawaban Soal 1



Gambar 8. Screenshot Hasil Tampilan Kunjungi (landscape) Jawaban Soal 1



Gambar 9. Screenshot Hasil Tampilan Favorit (landscape) Jawaban Soal 1



Gambar 10. Screenshot Hasil Tampilan Tanpa Internet (landscape) Jawaban Soal 1

C. Pembahasan

1. AndroidManifest.xml

- Baris 5–6:
Bagian ini mendeklarasikan izin (*permissions*) yang dibutuhkan oleh aplikasi. `android.permission.INTERNET` memberikan izin kepada aplikasi untuk mengakses internet, yang penting untuk mengambil data dari API online. `android.permission.ACCESS_NETWORK_STATE` memungkinkan aplikasi untuk memeriksa status koneksi jaringan, seperti apakah perangkat terhubung ke Wi-Fi atau data seluler.
- Baris 8–19:
Ini adalah blok utama `<application>` yang berisi konfigurasi global untuk seluruh aplikasi.
 - `android:name=".MakeupApp"`: Menentukan kelas `Application` kustom. Ini biasanya digunakan untuk inisialisasi awal saat aplikasi pertama kali dijalankan.
 - `android:icon`, `android:label`, `android:roundIcon`: Mengatur ikon dan nama aplikasi yang akan ditampilkan di peluncur (launcher) perangkat.
 - `android:theme="@style/Theme.Modul5"`: Menetapkan tema visual default untuk semua layar dalam aplikasi.
 - `android:usesCleartextTraffic="true"`: Izin ini mengizinkan aplikasi untuk berkomunikasi melalui protokol HTTP yang tidak terenkripsi. Izin ini sering kali diperlukan jika API yang digunakan belum mendukung HTTPS.
- Baris 20–28:
Bagian ini mendefinisikan `MainActivity`, yang merupakan layar utama aplikasi.

2. MakeupApp.kt

- Baris 9–10:
Di sini, kelas `MakeupApp` dideklarasikan sebagai turunan dari kelas `Application`. Menjadikannya kelas `Application` kustom memungkinkan inisialisasi komponen-komponen penting yang akan digunakan di seluruh siklus hidup aplikasi, bukan hanya pada satu `Activity`.
- Baris 11:
Variabel `database` diinisialisasi secara *lazy*. Artinya, objek `MakeupDatabase` (database Room) baru akan dibuat saat pertama kali diakses. Ini adalah praktik yang efisien untuk menghemat sumber daya karena database tidak langsung dimuat saat aplikasi dimulai.

`MakeupDatabase.getDatabase(this)` digunakan untuk mendapatkan instance tunggal (*singleton*) dari database.

- Baris 12–17:
Variabel `repository` juga diinisialisasi secara lazy. `MakeupRepository` adalah komponen arsitektur yang bertanggung jawab sebagai satu-satunya sumber data (*single source of truth*). Ia diberi dua dependensi `apiService` dari `RetrofitInstance` untuk mengambil data dari jaringan (remote) dan `makeupDao` dari database untuk mengakses data lokal (*cache*).
- Baris 19–21:
Variabel `connectivityObserver` diinisialisasi secara lazy untuk memantau status koneksi internet. Objek `NetworkConnectivityObserver` akan mengamati perubahan jaringan (terhubung atau tidak) di seluruh aplikasi, memungkinkan UI atau logika bisnis untuk bereaksi sesuai kondisi jaringan.

3. MakeupDao.kt

- Baris 9–10:
Anotasi `@Dao` menandai interface ini sebagai *Data Access Object* (DAO) untuk Room. DAO adalah komponen utama yang bertanggung jawab untuk mendefinisikan metode-metode yang akan digunakan untuk mengakses database aplikasi.
- Baris 12–13:
Fungsi `upsertAll` menggunakan anotasi `@Upsert`. Ini adalah operasi "*insert-or-update*" yang akan memasukkan daftar `MakeupEntity` baru ke dalam database. Jika ada produk dengan `primary key` yang sama sudah ada, maka data produk tersebut akan diperbarui. Fungsi ini ditandai sebagai *suspend* karena operasi tulis ke database harus dijalankan di luar thread utama, biasanya dalam sebuah coroutine.
- Baris 15–16
Fungsi `getAllMakeup` menggunakan anotasi `@Query` untuk menjalankan perintah SQL. Perintah `SELECT * FROM makeup_products` akan mengambil semua data dari tabel produk makeup dan mengurutkannya berdasarkan ID. Fungsi ini mengembalikan `Flow<List<MakeupEntity>>`, yang memungkinkan pengamatan data secara *real-time*. Setiap kali data di dalam tabel berubah, `Flow` ini akan secara otomatis mengirimkan daftar data terbaru ke pengamatnya (misalnya, UI).
- Baris 18–19:
Fungsi `getMakeupById` mengambil satu entitas makeup dari tabel berdasarkan `id` yang diberikan. Seperti `getAllMakeup`, fungsi ini juga mengembalikan `Flow`, sehingga UI dapat secara reaktif diperbarui jika data item tersebut berubah.

- Baris 21–22:
Fungsi `updateFavoriteStatus` menjalankan perintah `UPDATE` untuk mengubah status `isFavorite` dari sebuah produk. Fungsi ini memerlukan `id` produk dan status favorit yang baru (`true` atau `false`). Ini adalah fungsi `suspend` karena merupakan operasi tulis.
- Baris 24–25:
Fungsi `getFavoriteMakeup` mirip dengan `getAllMakeup`, tetapi dengan filter `WHERE isFavorite = 1`. Ini secara spesifik mengambil semua produk yang telah ditandai sebagai favorit oleh pengguna. Fungsi ini juga mengembalikan `Flow` untuk pembaruan reaktif.
- Baris 27–28:
Fungsi `getFavoriteIds` adalah query yang lebih efisien jika hanya daftar ID produk favorit yang dibutuhkan, bukan seluruh objek datanya. Ini adalah fungsi `suspend` untuk operasi baca satu kali.
- Baris 30–31:
Fungsi `clearAll` menjalankan perintah `DELETE` untuk menghapus semua data dari tabel `makeup_products`. Ini berguna untuk membersihkan cache lokal, dan merupakan fungsi `suspend` karena merupakan operasi tulis ke database.

4. MakeupEntity.kt

- Baris 5:
Anotasi `@Entity(tableName = "makeup_products")` menandai kelas ini sebagai sebuah tabel dalam database Room. `tableName` secara eksplisit mendefinisikan nama tabel yang akan dibuat, yaitu `makeup_products`.
- Baris 6:
`MakeupEntity` didefinisikan sebagai sebuah data class. Ini adalah cara ringkas di Kotlin untuk membuat kelas yang tujuan utamanya adalah untuk menyimpan data. Setiap parameter dalam konstruktornya akan menjadi sebuah kolom di dalam tabel database.
- Baris 7:
Anotasi `@PrimaryKey` digunakan untuk menandai properti `id` sebagai kunci utama (primary key) tabel. Ini memastikan bahwa setiap baris data (setiap produk makeup) memiliki pengenal yang unik dan tidak boleh ada duplikat.
- Baris 8–15:
Setiap properti di dalam data class ini seperti `brand`, `name`, `price`, dan seterusnya merepresentasikan sebuah kolom di dalam tabel `makeup_products`. Tanda tanya (?) pada tipe data `String?` menunjukkan bahwa kolom tersebut boleh berisi nilai null. Ini penting untuk menangani kasus di mana data dari API mungkin tidak lengkap. Kolom `isFavorite` adalah

Boolean yang digunakan untuk menyimpan status apakah suatu produk telah ditandai sebagai favorit oleh pengguna.

5. MakeupMapper.kt

- Baris 5–17:
Fungsi ekstensi `toEntity` mengubah sebuah `List<MakeupProduct>` (hasil dari API) menjadi sebuah `List<MakeupEntity>` (format yang bisa disimpan ke database Room). Fungsi ini sangat penting karena saat mengubah data, ia juga menentukan status `isFavorite` untuk setiap produk. Caranya adalah dengan memeriksa apakah `id` produk dari API ada di dalam daftar `favoriteIds` yang didapat dari database lokal. Ini memastikan status favorit tidak hilang saat data diperbarui dari internet.
- Baris 19–30:
Fungsi ekstensi `toModel` melakukan kebalikannya. Ia mengubah satu objek `MakeupEntity` (yang diambil dari database) menjadi objek `MakeupProduct`. Konversi ini diperlukan agar data dari database dapat digunakan secara konsisten oleh lapisan UI dan logika bisnis, yang umumnya bekerja dengan model `MakeupProduct`.
- Baris 32–34:
Fungsi ekstensi ini adalah versi jamak dari fungsi sebelumnya. Ia dengan mudah mengubah `List<MakeupEntity>` menjadi `List<MakeupProduct>` dengan memanggil fungsi `toModel()` pada setiap item di dalam daftar.

6. DataStoreManager.kt

- Baris 9:
Baris ini membuat sebuah instance `DataStore` untuk seluruh aplikasi. `preferencesDataStore(name = "settings")` menciptakan sebuah file penyimpanan preferensi bernama `"settings"`. Dengan mendefinisikannya sebagai properti ekstensi pada `Context`, kita memastikan hanya ada satu instance (*singleton*) yang digunakan di seluruh aplikasi.
- Baris 11:
Kelas `DataStoreManager` adalah kelas pembantu (*helper class*) yang menyediakan cara yang bersih dan terpusat untuk membaca dan menulis data ke `DataStore`.
- Baris 13:
Di dalam companion object, `DARK_MODE_KEY` didefinisikan menggunakan `booleanPreferencesKey("dark_mode")`. Ini adalah kunci yang aman secara tipe (*type-safe*) untuk mengakses nilai boolean di

DataStore. Menggunakan kunci seperti ini membantu menghindari kesalahan pengetikan dan memastikan kita selalu bekerja dengan tipe data yang benar.

- Baris 16–17:
Properti `darkMode` mengekspos status mode gelap sebagai `Flow<Boolean>`. `context.dataStore.data` adalah `Flow` yang akan mengirim pembaruan setiap kali ada data yang berubah di `DataStore`. Operator `.map` kemudian digunakan untuk mengambil nilai spesifik dari `DARK_MODE_KEY`. Jika kunci tersebut belum ada (misalnya saat aplikasi pertama kali dijalankan), nilai default `false` akan digunakan. Karena ini adalah `Flow`, UI dapat mengamatinya dan secara otomatis beralih antara mode terang dan gelap saat nilainya berubah.
- Baris 19–23:
Fungsi `setDarkMode` adalah fungsi `suspend` yang digunakan untuk menyimpan status mode gelap. Operasi tulis ke disk adalah operasi yang berpotensi lambat, sehingga harus dijalankan secara asinkron (dalam sebuah `coroutine`) agar tidak memblokir UI. `context.dataStore.edit { ... }` membuka sesi transaksi untuk mengubah data dengan aman.

7. MakeupDatabase.kt

- Baris 11:
Anotasi `@Database` mengonfigurasi kelas ini sebagai database `Room`.
 - `entities = [MakeupEntity::class]`: Memberi tahu `Room` bahwa tabel yang akan dikelola di dalam database ini adalah `MakeupEntity`.
 - `version = 2`: Menentukan versi skema database saat ini. Jika struktur tabel diubah, nomor versi ini harus dinaikkan.
 - `exportSchema = false`: Menonaktifkan ekspor skema database.
- Baris 12:
Kelas `MakeupDatabase` didefinisikan sebagai `abstract` dan mewarisi `RoomDatabase`. Implementasi konkret dari kelas ini akan dibuat secara otomatis oleh `Room`.
- Baris 13:
Fungsi `abstract makeupDao()` mendeklarasikan cara untuk mendapatkan akses ke `MakeupDao`. `Room` akan menyediakan implementasi untuk fungsi ini, yang akan mengembalikan instance dari DAO.
- Baris 15–31:
Blok `companion object` berisi logika untuk membuat instance database menggunakan pola singleton. Pola ini sangat penting karena memastikan bahwa hanya ada satu koneksi database yang dibuka selama aplikasi berjalan, sehingga menghemat sumber daya dan menghindari masalah konsistensi data.

- `@Volatile private var INSTANCE`: Variabel ini akan menyimpan satu-satunya instance database. Anotasi `@Volatile` memastikan bahwa nilainya selalu sinkron di antara semua thread.
- `fun getDatabase(context: Context)`: Ini adalah metode publik untuk mengakses database. Logika `INSTANCE ?: synchronized(this)` memastikan bahwa jika instance belum ada, instance baru akan dibuat di dalam blok yang aman untuk thread (*thread-safe*) untuk mencegah pembuatan ganda.
- `.fallbackToDestructiveMigration()`: Ini adalah strategi migrasi sederhana. Jika versi database ditingkatkan, Room akan menghapus data lama dan membuat database baru. Ini mudah diterapkan tetapi menyebabkan kehilangan data pengguna.

8. MakeupProduct.kt

- Baris 5:
Anotasi `@Serializable` memberi tahu compiler `kotlinx.serialization` bahwa kelas ini dapat di-serialisasi dan di-deserialisasi. Artinya, ia dapat dikonversi dari dan ke format seperti JSON.
- Baris 6:
`MakeupProduct` didefinisikan sebagai data class, yang ideal untuk menyimpan data. Setiap properti dalam konstruktornya mewakili sebuah field dalam respons JSON dari API.
- Baris 11, 12, dan 14:
Anotasi `@SerializedName` digunakan untuk memetakan nama properti di Kotlin ke nama field yang berbeda di dalam JSON. Contohnya, field `"image_link"` dalam JSON akan dipetakan ke properti `imageLink` di kelas Kotlin. Ini sangat berguna untuk mengikuti konvensi penamaan Kotlin (`camelCase`) sambil tetap kompatibel dengan nama field dari API (`snake_case`).
- Baris 15:
Properti `isFavorite` diberi nilai default `false`. Ini berarti saat objek `MakeupProduct` dibuat dari JSON (yang tidak memiliki field `isFavorite`), properti ini akan secara otomatis diinisialisasi dengan nilai `false`. Ini adalah nilai awal yang aman sebelum diperiksa dengan data lokal dari database.

9. MakeupApiService.kt

- Baris 6:
Anotasi `@GET("api/v1/products.json")` adalah instruksi untuk Retrofit. Ini menandakan bahwa fungsi di bawahnya akan melakukan sebuah permintaan HTTP GET ke endpoint spesifik, yaitu `"api/v1/products.json"`. URL

lengkapnya akan dibentuk dengan menggabungkan URL dasar (yang dikonfigurasi di `RetrofitInstance`) dengan endpoint ini.

- Baris 7:
Fungsi `getProducts` dideklarasikan sebagai `suspend`. Ini sangat penting karena operasi jaringan tidak boleh berjalan di thread utama (UI thread). Dengan `suspend`, pemanggilan fungsi ini dapat dijeda saat menunggu respons dari server dan dilanjutkan kembali setelah respons diterima, tanpa memblokir aplikasi. Fungsi ini akan mengembalikan `List<MakeupProduct>`, yang berarti Retrofit akan secara otomatis mengurai (parse) respons JSON dari API menjadi sebuah daftar objek `MakeupProduct`.

10. `RetrofitInstance.kt`

- Baris 11–14:
Di sini, instance `Json` dari `kotlinx.serialization` dikonfigurasi.
 - `ignoreUnknownKeys = true`: Mengizinkan Retrofit untuk mengabaikan field di JSON yang tidak ada di dalam data class `MakeupProduct`. Ini membuat aplikasi lebih tahan terhadap perubahan API.
 - `coerceInputValues = true`: Memaksa konversi nilai jika tipe data tidak cocok, misalnya jika API mengirimkan `null` untuk field yang tidak boleh `null`.
 - Baris 16–18: `HttpLoggingInterceptor` disiapkan untuk mencatat (log) detail dari setiap permintaan jaringan (request) dan responsnya (response) ke Logcat. `Level.BODY` berarti seluruh isi dari request dan response akan ditampilkan, yang sangat berguna untuk proses debugging.
- Baris 20–22:
`OkHttpClient` dibuat sebagai klien HTTP kustom. Klien inilah yang sebenarnya akan menangani koneksi. Logging interceptor yang sudah dibuat sebelumnya ditambahkan ke klien ini.
- Baris 24–30:
Blok ini adalah inti dari konfigurasi Retrofit.
 - `Retrofit.Builder()`: Memulai proses pembangunan instance Retrofit.
 - `.baseUrl(...)`: Menetapkan URL dasar untuk semua panggilan API. Setiap endpoint yang didefinisikan di `MakeupApiService` akan digabungkan dengan URL ini.
 - `.client(client)`: Menggunakan `OkHttpClient` kustom yang telah dikonfigurasi dengan logging.
 - `.addConverterFactory(...)`: Memberi tahu Retrofit untuk menggunakan `kotlinx.serialization` sebagai konverter. Ini akan

secara otomatis mengubah respons JSON dari server menjadi objek-objek Kotlin (`MakeupProduct`).

- Baris 32–34:
Retrofit membuat implementasi konkret dari interface `MakeupApiService`. Objek api inilah yang nantinya akan dipanggil dari `Repository` untuk benar-benar melakukan panggilan jaringan seperti `getProducts()`.

11. `MakeupRepository.kt`

- Baris 14–16:
Ini adalah konstruktor kelas yang menerima dua dependensi utama `apiService` untuk komunikasi dengan API dan `makeupDao` untuk akses ke database Room.
- Baris 17–28:
Fungsi `fetchAndCacheProducts` adalah inti dari logika sinkronisasi data.
 - Ia menggunakan `flow` untuk mengelola operasi asinkron dan mengirimkan hasilnya (`Result.success` atau `Result.failure`).
 - Pertama, ia memanggil API untuk mendapatkan data produk terbaru.
 - Kemudian, ia mengambil daftar ID produk yang sudah ditandai sebagai favorit dari database lokal.
 - Data dari API kemudian diubah menjadi `MakeupEntity` (menggunakan `toEntity`), dengan status favorit yang dipertahankan.
 - Terakhir, data yang sudah siap ini disimpan ke database menggunakan `upsertAll`.
 - Blok `try-catch` digunakan untuk menangani error jaringan atau pengecualian lainnya, lalu mengirimkan kegagalan tersebut ke pengamat.
- Baris 30–32:
Fungsi `getProductsFromCache` menyediakan akses ke semua data produk yang ada di database lokal. Ia mengambil `Flow<List<MakeupEntity>>` dari DAO dan menggunakan `.map` untuk mengubahnya menjadi `Flow<List<MakeupProduct>>`, yaitu model data yang dimengerti oleh lapisan UI.
- Baris 34–36:
Fungsi `getProductById` bekerja mirip seperti di atas, tetapi hanya untuk mengambil satu produk spesifik berdasarkan ID-nya.
- Baris 38–40:
Fungsi `getFavoriteProducts` secara spesifik mengambil daftar semua produk yang telah ditandai sebagai favorit dari database.
- Baris 42–44:

Fungsi `suspend toggleFavoriteStatus` dieksekusi ketika pengguna menekan tombol favorit. Ia memanggil DAO untuk memperbarui status `isFavorite` dari produk tertentu di dalam database.

12. ProductItem.kt

- Baris 30–37:

Fungsi `formatPrice` adalah fungsi pembantu yang mengubah harga dalam bentuk `String` menjadi format mata uang Rupiah yang mudah dibaca (misalnya, "Rp 150.000"). Fungsi ini menggunakan `NumberFormat` dengan lokal Indonesia (`id`, `ID`) untuk memastikan formatnya benar dan aman menangani nilai yang mungkin `null` atau tidak valid dengan blok `try-catch`.

- Baris 39–171:

Ini adalah fungsi `@Composable` utama bernama `ProductItem`. Fungsinya adalah untuk menampilkan data dari satu `MakeupProduct` ke dalam sebuah kartu (Card) UI yang interaktif.

- Baris 51–57:

Komponen `Card` digunakan sebagai wadah utama. Kartu ini dibuat bisa diklik (`clickable`) untuk membuka halaman detail, namun interaksi ini dinonaktifkan jika `isOnline` bernilai `false`.

- Baris 62–71:

`AsyncImage` dari pustaka `Coil` digunakan untuk memuat gambar produk dari URL secara asinkron. Ia juga menyediakan gambar placeholder yang akan ditampilkan saat gambar asli sedang dimuat atau jika terjadi error.

- Baris 73–133:

Komponen `Column` mengatur semua informasi teks seperti nama, tipe, harga, dan deskripsi produk secara vertikal. Teks dibatasi jumlah barisnya agar tata letak tetap rapi.

- Baris 135–156:

Dua buah `Button`, "Kunjungi" dan "Detail", ditempatkan di dalam sebuah `Row`. Tombol-tombol ini juga dinonaktifkan jika tidak ada koneksi internet untuk mencegah aksi yang tidak diinginkan.

- Baris 159–169:

`IconButton` dengan ikon hati ditempatkan di pojok kanan atas kartu. Ikon ini akan berubah (terisi atau kosong) tergantung pada status `isFavorite` produk dan memungkinkan pengguna untuk mengubah status favorit tersebut saat diklik.

13. Navigation.kt

- Baris 3:

`ROUTE_LIST` adalah sebuah konstanta `String` yang mendefinisikan nama rute untuk layar daftar produk.

- Baris 4:
`ARG_MAKEUP_ID` adalah konstanta yang mendefinisikan nama kunci (key) untuk argumen yang akan dikirimkan ke layar detail, yaitu ID produk.
- Baris 5:
`ROUTE_DETAIL` mendefinisikan pola rute untuk layar detail. Pola ini menyertakan placeholder `{makeupId}` yang nantinya akan diisi dengan ID produk yang sebenarnya saat navigasi terjadi.
- Baris 7–9:
Fungsi `createDetailRoute` adalah sebuah fungsi pembantu (*helper function*) yang sangat berguna. Ia mempermudah pembuatan rute lengkap ke layar detail dengan aman. Alih-alih menggabungkan string secara manual, kita cukup memanggil fungsi ini dan memberikan `makeupId`, dan fungsi ini akan mengembalikan rute yang sudah benar, misalnya `"makeup_detail/123"`.

14. `ApiMakeupItem.kt`

- Baris 18–25:
Komponen `Card` berfungsi sebagai kontainer utama dengan sudut membulat dan efek bayangan (`elevation`) untuk menonjolkan item.
- Baris 26–52:
Tata letak utama menggunakan `Row` untuk menyusun elemen secara horizontal.
 - `Image`:
Berada di sebelah kiri, komponen ini menggunakan `rememberAsyncImagePainter` dari pustaka `Coil` untuk memuat gambar produk dari URL (`product.imageUrl`) secara asinkron.
 - `Column`:
Berada di sebelah kanan gambar, komponen ini menyusun semua informasi teks (nama, tipe, harga, dan deskripsi) secara vertikal.
- Baris 53–83:
Bagian ini menampilkan detail teks produk. Setiap komponen `Text` diberi nilai default (seperti "Tanpa Nama" atau "-") untuk menangani kasus di mana data dari API mungkin kosong (`null`). Deskripsi produk dibatasi hingga 4 baris untuk menjaga tata letak tetap rapi.
- Baris 85–118:
Di bagian bawah `Column`, sebuah `Row` berisi dua `Button`: "Kunjungi" dan "Detail". Masing-masing tombol memiliki `onClick` handler yang akan menjalankan fungsi callback (`onVisitClick` dan `onDetailClick`) yang

diteruskan sebagai parameter saat tombol ditekan. Aksi klik juga dicatat ke Logcat untuk keperluan debugging.

15. **MakeupItem.kt**

- Baris 23–30:
Komponen `Card` berfungsi sebagai wadah utama dengan sudut membulat dan efek bayangan, memberikan tampilan yang rapi dan terstruktur untuk setiap item.
- Baris 31–60:
Tata letak utama menggunakan `Row` untuk menempatkan gambar di sebelah kiri dan teks di sebelah kanan.
 - `AsyncImage`: Menggunakan pustaka `Coil` untuk memuat gambar produk dari URL secara asinkron. Komponen ini juga menampilkan gambar error bawaan Android jika gagal memuat.
 - `Spacer`: Memberikan jarak horizontal antara gambar dan kolom teks.
- Baris 62–93:
Komponen `Column` mengatur semua informasi teks secara vertikal.
 - `Text`: Menampilkan nama produk, merek, tipe produk, dan deskripsi. Terdapat logika untuk menangani nilai `null` dengan memberikan teks pengganti. Deskripsi dibatasi hingga 4 baris untuk mencegah teks yang terlalu panjang.
 - Baris 95–129: Di bagian bawah, sebuah `Row` berisi dua `Button`: "Kunjungi" dan "Detail".
 - Setiap tombol memanggil fungsi lambda (`onVisitClick` dan `onDetailClick`) saat ditekan, mengirimkan data yang relevan seperti `imageLink` atau `id produk`.
 - Terdapat `Log.d` di dalam `onClick` untuk mencatat aktivitas klik ke Logcat, yang sangat membantu saat proses debugging.

16. **MakeupListScreen.kt**

- Baris 42–45:
Di sini, `ViewModel`, `Repository`, dan `ConnectivityObserver` diinisialisasi. Komponen-komponen ini diambil dari instance `MakeupApp`, yang berfungsi sebagai penyedia dependensi pusat untuk aplikasi.
- Baris 47–48:
Status koneksi jaringan (`isOnline`) diamati menggunakan `collectAsState`. UI akan secara otomatis diperbarui (`recompose`) setiap kali status koneksi berubah.
- Baris 50–52:

`uiState` dari `ViewModel` dikumpulkan untuk mendapatkan data produk, favorit, status loading, dan pesan error. `selectedTabIndex` adalah state lokal untuk mengelola tab mana yang sedang aktif.

- Baris 55–66:
`LaunchedEffect` digunakan untuk menangani event satu kali dari `ViewModel`. Ini adalah cara yang bersih untuk memisahkan logika navigasi atau aksi lain (seperti membuka URL) dari logika utama UI. Ketika `ViewModel` mengirimkan event, `LaunchedEffect` akan menanganinya, misalnya dengan memanggil `navController.navigate()` atau memulai `Intent` baru.
- Baris 68–95:
Komponen `Scaffold` menyediakan struktur dasar layar dengan `AppBar` dan area konten. Di dalam `Column`, sebuah `TabRow` dibuat untuk memungkinkan pengguna beralih antara tampilan "Home" dan "Favorit".
- Baris 97–117:
Ini adalah komponen visual yang sangat penting untuk user experience. Sebuah `Row` akan ditampilkan di bawah tab jika `isOnline` bernilai `false`, memberikan umpan balik yang jelas kepada pengguna bahwa mereka sedang offline.
- Baris 119–133:
Blok `when` digunakan untuk menampilkan konten yang sesuai berdasarkan `selectedTabIndex`. Ia memanggil `Composable ProductList` dengan data yang benar (semua produk untuk tab "Home", dan produk favorit untuk tab "Favorit").

`ProductList Composable`

Ini adalah komponen UI yang dapat digunakan kembali, bertugas khusus untuk menampilkan daftar produk.

- Baris 144–165:
Komponen `Box` ini secara cerdas mengatur apa yang harus ditampilkan berdasarkan state yang diterima:
 - Jika `isLoading` dan daftar produk kosong, ia menampilkan `CircularProgressIndicator` di tengah.
 - Jika ada error, ia menampilkan pesan error.
 - Jika daftar produk kosong (dan tidak loading/error), ia menampilkan `emptyMessage`.
 - Jika ada data, ia menampilkan `LazyColumn`.
- Baris 156–165:
`LazyColumn` adalah cara paling efisien untuk menampilkan daftar yang panjang. Ia hanya merender item yang terlihat di layar. `key = { it.id }` adalah optimasi penting untuk membantu `Compose` mengidentifikasi item secara unik,

meningkatkan performa. Untuk setiap `product` di dalam daftar, ia memanggil `ProductItem` dan meneruskan data serta fungsi callback dari `ViewModel` untuk menangani klik.

17. `MakeupDetailScreen.kt`

- Baris 48–50:
Sama seperti layar sebelumnya, `ViewModel` dan `Repository` diinisialisasi untuk mendapatkan akses ke logika bisnis dan data.
- Baris 52–54:
`LaunchedEffect(key1 = makeupId)` adalah blok yang sangat penting. Ini akan berjalan secara otomatis setiap kali nilai `makeupId` berubah. Di dalamnya, ia memanggil `viewModel.getProductDetails(makeupId)` untuk memerintahkan `ViewModel` agar mengambil data detail produk yang spesifik dari `repository`.
- Baris 56:

```
val product by  
viewModel.detailProduct.collectAsStateWithLifecycle()
```


mengamati `Flow` yang berisi detail produk dari `ViewModel`. Variabel `product` akan awalnya `null`, dan UI akan diperbarui secara otomatis (`recompose`) setelah data produk berhasil diambil.
- Baris 58–92:
Komponen `Scaffold` digunakan untuk struktur layar.
 - `TopAppBar`: Dibuat transparan dan berisi dua aksi penting. Pertama, `navigationIcon` (tombol panah kembali) yang memanggil `navController.popBackStack()` untuk kembali ke layar daftar. Kedua, `IconButton` di bagian `actions` (pojok kanan atas) untuk menandai favorit. Ikon ini hanya muncul jika data `product` sudah ada, dan tampilannya (hati terisi atau kosong) dikendalikan oleh properti `isFavorite` dari produk.
- Baris 93–175:
Ini adalah area konten utama.
 - Sebuah `Box` digunakan untuk menampilkan `CircularProgressIndicator` di tengah layar jika `product` masih `null` (artinya data sedang dimuat).
 - Setelah data tersedia, sebuah `Column` yang bisa di-scroll (`verticalScroll`) akan ditampilkan. Kolom ini berisi semua detail produk seperti gambar (`AsyncImage`), nama, tipe, dan deskripsi lengkap, yang diatur dengan rapi menggunakan `Spacer` dan `padding`.

- Di bagian paling bawah, terdapat sebuah Button "Kembali" yang juga berfungsi untuk menutup layar detail ini.

18. MakeupViewModel.kt

- Baris 11–16:
data class MakeupListUiState mendefinisikan semua data yang dibutuhkan oleh UI untuk layar daftar. Ini mencakup status isLoading, daftar semua produk (products), daftar favorit, dan pesan error. Menggabungkan semua state dalam satu objek seperti ini mempermudah pengelolaan.
- Baris 21 & 24:
_uiState dan _detailProduct adalah MutableStateFlow. Ini adalah state holder yang menyimpan data UI dan dapat diperbarui. Versi publiknya (uiState dan detailProduct) diekspos sebagai StateFlow (read-only) agar UI bisa mengamatinya secara reaktif.
- Baris 26–27:
_eventFlow adalah MutableSharedFlow. Ini digunakan untuk mengirim event atau aksi satu kali (seperti navigasi atau membuka URL) dari ViewModel ke UI, tanpa menyimpannya sebagai state.
- Baris 29–33:
Blok init dieksekusi saat ViewModel pertama kali dibuat. Ia langsung memulai proses pengamatan data dari cache (observeProductsFromCache dan observeFavorites) dan memicu penyegaran data dari jaringan (refreshProducts).
- Baris 35–44:
Fungsi refreshProducts memanggil repository untuk mengambil data terbaru dari API dan menyimpannya ke cache. Selama proses ini, ia memperbarui _uiState untuk menampilkan indikator isLoading dan menangani kemungkinan error jaringan.
- Baris 46–58:
Fungsi observeProductsFromCache dan observeFavorites secara terus-menerus mengamati perubahan data di database melalui Flow dari repository. Setiap kali ada perubahan (misalnya, setelah penyegaran data atau perubahan status favorit), _uiState akan otomatis diperbarui dengan data terbaru.
- Baris 60–66:
Fungsi getProductDetails dipanggil saat layar detail dibuka. Ia meminta repository untuk mengambil data satu produk spesifik berdasarkan ID-nya dan memperbarui _detailProduct.
- Baris 68–78:

Fungsi `toggleFavorite` berisi logika untuk mengubah status favorit. Ia mencari produk yang relevan (baik dari state layar detail maupun layar daftar), lalu memanggil repository untuk memperbarui statusnya di database.

- Baris 80–95:
Fungsi `onDetailClicked` dan `onVisitClicked` adalah event handler. Saat tombol di UI ditekan, fungsi ini akan mengirim `UiEvent` yang sesuai ke `_eventFlow`. UI kemudian akan menangkap event ini dan melakukan aksi yang diperlukan (navigasi atau membuka link).
- Baris 97–100:
`sealed class UiEvent` adalah cara yang aman untuk mendefinisikan berbagai jenis aksi satu kali yang bisa dikirim oleh `ViewModel` ke UI.

19. `MakeupViewModelFactory.kt`

- Baris 6:
Kelas `MakeupViewModelFactory` mewarisi `ViewModelProvider.Factory`, yang mengharuskannya untuk meng-override satu metode, yaitu `create`.
- Baris 7–12:
Metode `create` adalah inti dari factory ini.
 - Ia memeriksa apakah kelas `ViewModel` yang diminta (`modelClass`) adalah `MakeupViewModel` atau subclassesnya menggunakan `isAssignableFrom`.
 - Jika benar, ia akan membuat instance baru dari `MakeupViewModel` dengan meneruskan repository yang telah diterima oleh factory ini. `@SuppressWarnings("UNCHECKED_CAST")` digunakan untuk menekan peringatan dari compiler karena proses casting ini sudah dipastikan aman oleh pengecekan `if` sebelumnya.
 - Jika kelas yang diminta bukan `MakeupViewModel`, ia akan melempar `IllegalArgumentException`, yang menandakan bahwa factory ini tidak tahu cara membuat `ViewModel` jenis lain.

20. `MainActivity.kt`

- Baris 23–38:
Ini adalah kelas `MainActivity`, yang merupakan satu-satunya `Activity` dalam aplikasi ini.
 - Dalam metode `onCreate`, beberapa pengaturan awal dilakukan. `enableEdgeToEdge()` memungkinkan aplikasi untuk ditampilkan di seluruh layar, termasuk di belakang status bar dan navigation bar.

- `setContent` adalah fungsi utama dari Jetpack Compose yang digunakan untuk mendefinisikan seluruh antarmuka pengguna (UI) aplikasi.
- Di dalam `setContent`, `Modul5Theme` diterapkan untuk memberikan tema visual (warna, tipografi) yang konsisten di seluruh aplikasi.
- Terakhir, `Composable MainApp()` dipanggil untuk membangun dan mengelola struktur navigasi aplikasi.
- Baris 42–43:
`Composable MainApp` adalah inti dari sistem navigasi. Di sini, sebuah `NavController` dibuat menggunakan `rememberNavController()`. `NavController` ini bertanggung jawab untuk mengatur perpindahan antar layar (screen) dan mengelola tumpukan navigasi (back stack).
- Baris 45–48:
 Komponen `NavHost` bertindak sebagai wadah yang akan menampilkan `Composable` yang sesuai berdasarkan rute saat ini. Ia dihubungkan dengan `NavController` dan diberi `startDestination` ke `Navigation.ROUTE_LIST`, yang berarti layar pertama yang akan ditampilkan saat aplikasi dibuka adalah `MakeupListScreen`.
- Baris 49–51:
`composable(Navigation.ROUTE_LIST)` mendefinisikan rute pertama. Ketika `NavController` menavigasi ke rute ini, konten yang akan ditampilkan adalah `MakeupListScreen`, dengan `NavController` yang diteruskan sebagai parameter agar layar tersebut juga dapat melakukan navigasi.
- Baris 52–60:
`composable(route = Navigation.ROUTE_DETAIL, ...)` mendefinisikan rute untuk layar detail.
 - Rute ini didefinisikan untuk menerima sebuah argumen, yaitu `makeupId`, yang tipenya ditentukan sebagai `NavType.IntType`.
 - Ketika navigasi ke rute ini terjadi, nilai `makeupId` diambil dari `BackStackEntry.arguments`.
 - Nilai `makeupId` tersebut kemudian diteruskan ke `Composable MakeupDetailScreen`, sehingga layar detail tahu produk mana yang harus ditampilkan.

21. Color.kt

- Baris 3–5:
 Mendefinisikan warna-warna terang dan lembut seperti `SoftPink`, `LightLilac`, dan `BlushPink`. Warna-warna ini kemungkinan besar digunakan sebagai warna latar belakang, permukaan, atau warna sekunder dalam tema aplikasi.
- Baris 7–9:

Mendefinisikan warna-warna yang lebih kuat dan gelap seperti Rose, Plum, dan Mauve. Warna-warna ini biasanya digunakan sebagai warna primer (utama), aksen, atau warna teks untuk menciptakan kontras.

22. Theme.kt

- Baris 10–20:
Blok `DarkColorScheme` mendefinisikan palet warna yang akan digunakan saat aplikasi berada dalam mode gelap (dark mode). Setiap properti, seperti `primary`, `secondary`, dan `background`, diberi nilai warna spesifik yang telah dideklarasikan di file `Color.kt`.
- Baris 22–32:
Blok `LightColorScheme` mendefinisikan palet warna untuk mode terang (light mode). Ini memastikan aplikasi memiliki tampilan yang konsisten dan mudah dibaca dalam kondisi pencahayaan normal.
- Baris 34–50:
Ini adalah fungsi Composable utama, `Modul5Theme`, yang bertugas untuk "membungkus" seluruh UI aplikasi dengan tema yang telah ditentukan.
 - Parameter `darkTheme: Boolean = isSystemInDarkTheme()` secara cerdas mendeteksi apakah perangkat pengguna sedang dalam mode gelap atau terang, dan menggunakan nilai tersebut sebagai default.
 - Blok `when` adalah logika inti di sini. Ia memilih skema warna yang sesuai (`DarkColorScheme` atau `LightColorScheme`) berdasarkan kondisi `darkTheme`.
 - Terakhir, ia memanggil komponen `MaterialTheme`, meneruskan `colorScheme` yang telah dipilih dan `Typography` (gaya teks). Semua Composable yang berada di dalam `content` dari `Modul5Theme` akan secara otomatis mewarisi dan menggunakan gaya visual ini.

23. Type.kt

File ini mendefinisikan semua gaya teks (tipografi) yang akan digunakan di seluruh aplikasi. Dengan mengaturnya di satu tempat, aplikasi akan memiliki tampilan teks yang konsisten dan mudah dikelola.

Objek `Typography` ini berisi berbagai definisi `TextStyle` untuk peran yang berbeda:

- `bodyLarge`: Gaya teks standar untuk paragraf atau konten utama, dengan ukuran dan ketebalan normal.
- `titleLarge`: Gaya teks untuk judul utama layar, dibuat lebih besar dan tebal (Bold) agar menonjol.

- `titleMedium`: Digunakan untuk subjudul atau judul bagian, sedikit lebih kecil dari `titleLarge`.
- `bodyMedium` & `bodySmall`: Gaya teks untuk informasi sekunder atau teks yang lebih kecil seperti keterangan (caption), dengan ukuran font yang lebih kecil.

Setiap `TextStyle` menentukan properti seperti `fontFamily` (jenis huruf), `fontWeight` (ketebalan), `fontSize` (ukuran), dan `lineHeight` (jarak antar baris) untuk memastikan keterbacaan dan hierarki visual yang jelas.

24. ConnectivityObserver.kt

- Baris 13:
Interface `ConnectivityObserver` mendefinisikan sebuah kontrak. Ia menetapkan bahwa setiap kelas yang ingin berfungsi sebagai pengamat konektivitas harus menyediakan satu metode, yaitu `observe()`, yang mengembalikan `Flow<Boolean>`.
- Baris 17:
Kelas `NetworkConnectivityObserver` adalah implementasi konkret dari interface di atas.
- Baris 20–21:
Mendapatkan `ConnectivityManager` dari sistem Android. Ini adalah komponen inti yang digunakan untuk berinteraksi dengan status jaringan.
- Baris 23–45:
Metode `observe()` adalah bagian utama dari kelas ini.
 - Ia menggunakan `callbackFlow`, sebuah builder `Flow` yang dirancang khusus untuk mengubah API berbasis callback (seperti `NetworkCallback`) menjadi `Flow` yang reaktif.
 - Di dalamnya, sebuah `NetworkCallback` dibuat. `onAvailable` akan dipanggil oleh sistem saat jaringan terhubung, dan `onLost` dipanggil saat jaringan terputus. Keduanya mengirimkan nilai `true` atau `false` ke `Flow`.
 - Sebelum mendaftarkan callback, ia memeriksa status koneksi awal dan langsung mengirimkannya ke `Flow`.
 - `connectivityManager.registerDefaultNetworkCallback(callback)` mulai mendengarkan perubahan status jaringan.
 - Blok `awaitClose` sangat penting. Blok ini akan dieksekusi secara otomatis saat pengamat tidak lagi dibutuhkan. Fungsinya adalah untuk membatalkan pendaftaran callback (`unregisterNetworkCallback`), yang mencegah kebocoran memori (memory leak).

- `.distinctUntilChanged()` adalah optimasi yang memastikan Flow hanya akan mengirimkan nilai baru jika statusnya benar-benar berubah (misalnya dari `true` ke `false`), bukan mengirim nilai yang sama berulang kali.

25. `build.gradle.kts` (Module App)

- Bagian `plugins`

Blok ini mendeklarasikan plugin-plugin yang dibutuhkan untuk membangun aplikasi:

- `com.android.application`: Plugin standar untuk membangun aplikasi Android.
- `org.jetbrains.kotlin.android`: Mengaktifkan dukungan untuk bahasa pemrograman Kotlin.
- `org.jetbrains.kotlin.kapt`: Plugin untuk memproses anotasi (seperti yang digunakan oleh Room).
- `kotlinx-serialization`: Mengaktifkan pustaka serialisasi Kotlin untuk mengubah data JSON menjadi objek.
- `org.jetbrains.kotlin.plugin.compose`: Plugin compiler khusus untuk Jetpack Compose.

- Bagian `android`

Blok ini berisi semua konfigurasi spesifik untuk aplikasi Android Anda:

- `namespace` & `applicationId`: Pengenal unik untuk aplikasi Anda.
- `compileSdk`, `minSdk`, `targetSdk`: Menentukan versi Android API yang digunakan untuk kompilasi, versi minimum yang didukung, dan versi target.
- `buildTypes`: Mengonfigurasi varian build. Di sini, `release` dikonfigurasi, meskipun `isMinifyEnabled = false` berarti kode tidak akan diperkecil (minify).
- `compileOptions` & `kotlinOptions`: Menetapkan versi Java dan Kotlin yang akan digunakan.
- `buildFeatures { compose = true }`: Secara eksplisit mengaktifkan Jetpack Compose untuk proyek ini.
- `packaging`: Mengatur cara pengepakan sumber daya. Di sini, ia digunakan untuk mengecualikan file metadata yang bisa menyebabkan konflik.

- Bagian `dependencies`

Ini adalah daftar semua pustaka eksternal yang digunakan oleh aplikasi Anda, dikelompokkan berdasarkan fungsinya:

- **Inti & UI (Jetpack Compose):** Pustaka dasar untuk Android, siklus hidup (lifecycle), dan semua komponen UI dari Jetpack Compose (seperti `material3`, `icons`, `ui-tooling`).
- **Navigasi:** `navigation-compose` untuk mengelola perpindahan antar layar di Compose.
- **ViewModel & State:** Pustaka untuk menggunakan `ViewModel` dan mengamati state secara reaktif di Compose.
- **Jaringan (Networking):**
 - `retrofit`: Pustaka untuk melakukan panggilan API.
 - `retrofit2-kotlinx-serialization-converter`: Konverter agar Retrofit bisa bekerja dengan `kotlinx.serialization`.
 - `logging-interceptor`: Untuk mencatat (log) detail panggilan API, sangat berguna untuk debugging.
- **Pemuatan Gambar (Image Loading):** `coil-compose` untuk memuat gambar dari internet secara efisien.
- **Penyimpanan Lokal (Local Storage):**
 - `room-runtime`, `room-compiler`, `room-ktx`: Pustaka Room untuk database SQLite lokal. kapt digunakan untuk memproses anotasi Room.
 - `datastore-preferences`: Jetpack `DataStore` untuk menyimpan preferensi sederhana.
- **Coroutines:** `kotlinx-coroutines-android` untuk mengelola tugas-tugas asinkron.
- **Pengujian (Testing):** Pustaka-pustaka untuk melakukan pengujian unit dan UI.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rrdtlsh/Praktikum_Mobile